

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ
Khoa Công nghệ thông tin



BÁO CÁO
CÁC VẤN ĐỀ HIỆN ĐẠI TRONG CÔNG NGHỆ THÔNG TIN

BÀI BÁO: PRACTICAL, AUTOMATED
SCENARIO-BASED MOBILE APP TESTING

Giảng viên: ThS. Nguyễn Đức Anh

Lớp học phần: INT3507_7

Nhóm sinh viên thực hiện: Nhóm 3

Trần Diệu Anh 21020279

Đinh Thị Mai Linh 21020773

Nguyễn Thị Hồng Hạnh 21020016

Hà Nội, tháng 12/2024

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ
Khoa Công nghệ thông tin



BÁO CÁO
CÁC VẤN ĐỀ HIỆN ĐẠI TRONG CÔNG NGHỆ THÔNG TIN

BÀI BÁO: PRACTICAL, AUTOMATED
SCENARIO-BASED MOBILE APP TESTING

Giảng viên:	ThS. Nguyễn Đức Anh
Lớp học phần:	INT3507_7
Nhóm sinh viên thực hiện:	Nhóm 3
Trần Diệu Anh	21020279
Đinh Thị Mai Linh	21020773
Nguyễn Thị Hồng Hạnh	21020016

Hà Nội, tháng 12/2024

PHÂN CÔNG CÔNG VIỆC

MSV	Họ và tên	Công việc đảm nhận	Phần trăm (%)
21020279	Trần Diệu Anh	- Dịch full bài báo - Tóm tắt phần V-Thực nghiệm + Kết luận - Làm Slide phần Thực nghiệm, Demo, Kết luận - Setup và run Demo - Viết báo cáo phần Thực nghiệm, Cài đặt và Demo, Phụ lục	34
21020773	Đinh Thị Mai Linh	- Dịch full bài báo - Tóm tắt phần I-Đặt vấn đề, III.B,C-Các mối quan hệ, IV-Kiểm thử trên ứng dụng - Làm Slide phần Background, Kiểm thử ứng dụng, Phương pháp đề xuất - Setup code - Viết báo cáo phần Background, Process-Sinh ca kiểm thử, Kết luận	33
21020016	Nguyễn Thị Hồng Hạnh	- Dịch full bài báo - Tóm tắt phần II-Example, III.A- Trích xuất các thực thể - Làm Slide phần Đặt vấn đề + Xây dựng đồ thị EKG - Setup code - Viết báo cáo phần Đặt vấn đề, Phương pháp đề xuất, Process-Xây dựng đồ thị EKG	33

MỤC LỤC

PHÂN CÔNG CÔNG VIỆC	2
DANH SÁCH HÌNH ẢNH.....	5
DANH SÁCH BẢNG BIỂU	6
I. Đặt vấn đề.....	7
1. Bối cảnh hiện nay	7
2. Các vấn đề tồn đọng	8
II. Phương pháp đề xuất	9
III. Background.....	11
1. Event Knowledge Graph - EKG.....	11
2. 4 loại thực thể	11
2.1. Text (TXT)	12
2.2. Widget (WID).....	12
2.3. Operation (OPT).....	12
2.4. Content (CNT).....	12
3. 5 mối quan hệ	13
3.1. TXT-TXT	13
3.2. CNT-OPT	13
3.3. CNT-TXT	13
3.4. CNT-WID	13
3.5. CNT-CNT	13
4. Giải quyết đồng tham chiếu.....	14
IV. Process	15
1. Xây dựng cây đồ thị EKG	15
1.1. Trích xuất thực thể	16
1.2. Xác định mối quan hệ.....	17
1.3. Giải quyết đồng tham chiếu	18
2. Kiểm thử ứng dụng di động dựa trên kịch bản.....	19
2.1. Phân tích GUI ứng dụng.....	20
2.2. Sinh ca kiểm thử dựa trên kịch bản	21
V. Research Question - Thực nghiệm bài báo	23

1. Đối tượng thử nghiệm	23
2. RQ1: Tính đúng đắn	24
3. RQ2: Độ tin cậy	25
4. RQ3: Độ chính xác	26
5. RQ4: Tính đầy đủ	27
6. RQ5: Tính hữu ích	28
VI. Cài đặt và Demo	30
1. Cài đặt	30
2. Cấu trúc và sử dụng đồ thị EKG	31
2.1. Neo4j	31
2.2. Logic của việc tạo đồ thị tri thức	32
2.3. Các bước truy vấn đồ thị	33
3. Demo	33
3.1. Xây dựng đồ thị EKG	33
3.2. Kiểm thử ứng dụng di động qua hình ảnh	35
VII. Kết luận	37
1. Điểm hay	37
2. Điểm cần cải thiện	37
VIII. Phụ lục	38
<i>Câu hỏi: Truy vấn là gì? Làm thế nào để có thể truy vấn trong biểu đồ EKG?</i>	38
Tài liệu tham khảo – Nguồn	40

DANH SÁCH HÌNH ẢNH

Hình 1: Sự phát triển của ứng dụng di động	7
Hình 2: ScenTest Framework.....	10
Hình 3: Ví dụ về đồ thị EKG.....	11
Hình 4: 4 loại thực thể và 5 mối quan hệ được SCENTEST định nghĩa trong EKG	12
Hình 5: Ví dụ đồng tham chiếu	14
Hình 6: Thuật toán 1: Xây dựng đồ thị EKG	15
Hình 7: Process Xây dựng đồ thị EKG	16
Hình 8: Ví dụ từ báo cáo kiểm thử cộng đồng	17
Hình 9: Process Kiểm thử trên ứng dụng di động.....	19
Hình 10: Thuật toán Kiểm thử ứng dụng dựa trên kịch bản	19
Hình 11: Công thức xác suất một widget được chọn	22
Hình 12: Công thức đánh giá độ chính xác GenRate.....	26
Hình 13: Công thức đánh giá tính đầy đủ ScenCov	28
Hình 14: File chứa các package cần cài đặt – requirements.txt	30
Hình 15: Config database cho Neo4j	31
Hình 16: Start Neo4j	31
Hình 17: Hàm analyze()	33
Hình 18: Quá trình run hàm analyze()	33
Hình 19: Kết quả sau khi run hàm analyze().....	34
Hình 20: Hàm connect()	34
Hình 21: Quá trình run hàm connect()	34
Hình 22: Hàm generate().....	35
Hình 23: Quá trình run hàm generate() – Thêm node vào đồ thị	35
Hình 24: Kết quả xây dựng cây đồ thị EKG	35
Hình 25: Ví dụ về state ban đầu muốn test	36
Hình 26: Hàm test tìm next_steps	36
Hình 27: Kết quả sau khi truy vấn.....	36
Hình 28: Hàm truy vấn node tiếp theo	39

DANH SÁCH BẢNG BIỂU

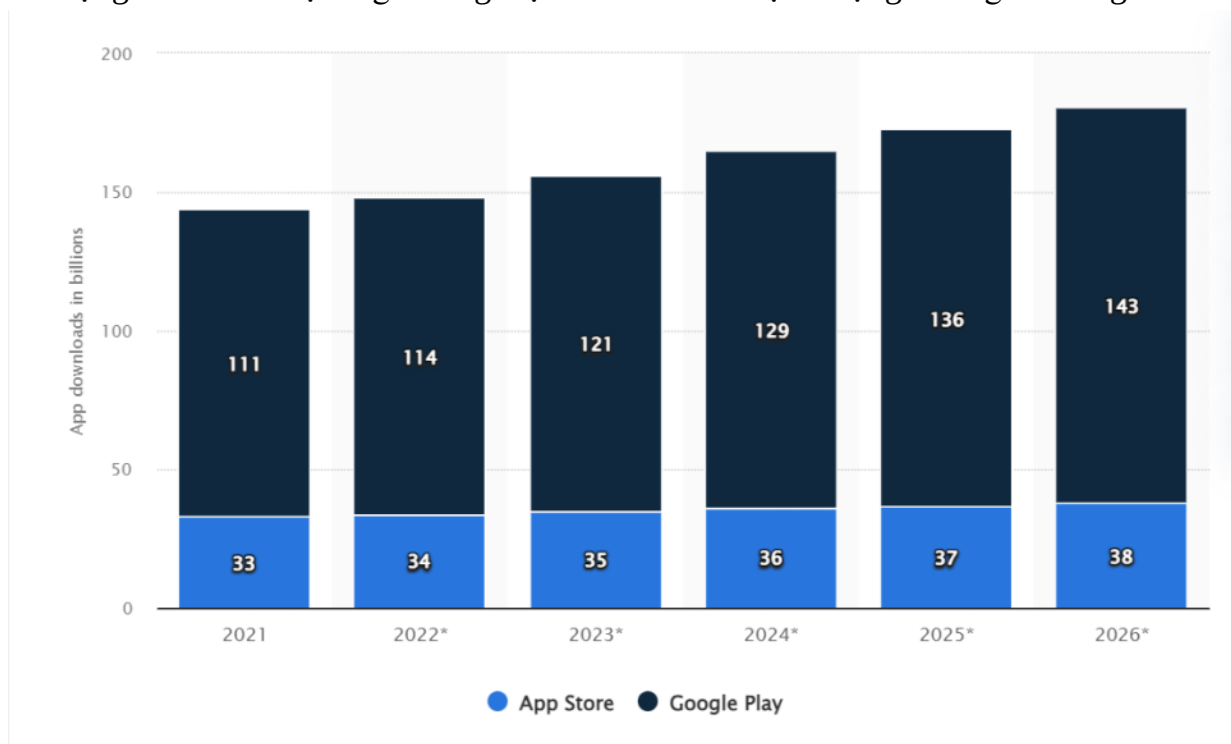
Bảng 1: Thống kê số lượng ứng dụng, event, scenario kiểm thử.....	24
Bảng 2: Thống kê các Entity và Relationship trong đồ thị EKG.....	24
Bảng 3: Kết quả của RQ1 – Tính đúng đắn	25
Bảng 4: Kết quả của RQ2 – Độ tin cậy	26
Bảng 5: Kết quả của RQ3 – Độ chính xác	27
Bảng 6: Kết quả của RQ4 – Tính đầy đủ	28
Bảng 7: Kết quả của RQ5 – Tính hữu ích	29

I. ĐẶT VẤN ĐỀ

1. Bối cảnh hiện nay

Với sự phát triển mạnh mẽ của Internet di động và công nghệ hiện nay, có thể thấy số lượng ứng dụng di động phát triển ngày càng nhanh, đặc biệt là khi nhu cầu sử dụng các thiết bị di động ngày càng tăng trên mọi nơi. Dự báo đến năm 2025, số lượng người dùng di động toàn cầu sẽ đạt khoảng 7,49 tỷ người. Theo báo cáo của Business of Apps, hiện tại đã có khoảng 3,3 triệu ứng dụng Android có sẵn trên Google Play Store, nó phản ánh rõ sự bùng nổ của các ứng dụng di động trong những năm gần đây.

Đặc biệt, vào tháng 2 năm 2024, gần 52 nghìn ứng dụng di động mới đã được phát hành, cho thấy tốc độ phát triển của thị trường này không ngừng gia tăng. Dự báo đến năm 2026, số lượt tải xuống ứng dụng di động sẽ đạt khoảng 143 tỷ, tăng gần 30% so với 111 tỷ lượt tải xuống được ghi nhận vào năm 2021. Điều này cho thấy ứng dụng di động không chỉ phát triển về số lượng mà còn có sự tăng trưởng mạnh mẽ về mức độ sử dụng của người dùng.



Hình 1: Sự phát triển của ứng dụng di động

Tuy nhiên, sự tăng trưởng nhanh chóng của thị trường ứng dụng di động cũng đi kèm với một số vấn đề đáng lo ngại. Một trong những vấn đề quan trọng nhất là chất lượng ứng dụng. Theo thống kê, người dùng phát hiện tới 44% lỗi trong các ứng dụng, điều này cho thấy sự thiếu sót trong quá trình kiểm thử và đảm bảo chất lượng. Thêm vào đó, 47% ứng dụng cần thêm thời gian thử nghiệm để hoàn thiện, và 58% người dùng cảm thấy thất vọng vì giao diện ứng dụng không nhất quán, ảnh hưởng đến trải nghiệm người dùng. Những con số này cho thấy rằng mặc dù số lượng ứng dụng ngày càng tăng, nhưng chất lượng của chúng vẫn là một vấn đề nghiêm trọng cần được giải quyết.

Chính vì vậy, yêu cầu về đảm bảo chất lượng ứng dụng đã trở nên khắt khe hơn bao giờ hết. Các nhà phát triển phải đối mặt với thách thức lớn khi không chỉ phát triển và cập nhật ứng dụng một cách nhanh chóng mà còn phải đảm bảo rằng các ứng dụng này đáp ứng được các tiêu chuẩn chất lượng cao, từ việc kiểm thử chức năng, giao diện đến khả năng hoạt động ổn định trên các thiết bị và nền tảng khác nhau. Việc đảm bảo chất lượng ứng dụng giờ đây không chỉ là một yêu cầu bắt buộc mà còn là yếu tố sống còn đối với sự thành công của bất kỳ ứng dụng di động nào trong thị trường cạnh tranh khốc liệt này.

2. Các vấn đề tồn đọng

Mặc dù các phương pháp kiểm thử tự động hiện nay đã đạt được những bước tiến đáng kể, nhưng vẫn tồn tại một số vấn đề cần được giải quyết để cải thiện hiệu quả và độ chính xác của quá trình kiểm thử.

Thứ nhất, các phương pháp kiểm thử tự động hiện tại vẫn phụ thuộc vào kỹ năng của các nhà phát triển ứng dụng. Các nhà phát triển thường phải tạo ra các tập lệnh kiểm thử dựa trên các framework kiểm thử và sử dụng công nghệ ghi và phát lại (record and replay). Tuy nhiên, việc này đòi hỏi nhiều sự can thiệp thủ công và có sự phụ thuộc vào kỹ năng của các nhà phát triển. Do đó, quá trình này vẫn tiêu tốn nhiều nhân lực và thời gian, gây trở ngại lớn đối với việc triển khai kiểm thử tự động một cách hiệu quả và toàn diện.

Thứ hai, mặc dù các phương pháp kiểm thử như kiểm thử ngẫu nhiên, kiểm thử dựa trên mô hình, hay học sâu/học tăng cường đã được áp dụng, mục tiêu chính của chúng thường chỉ tập trung vào việc tối ưu hóa độ bao phủ mã nguồn. Tuy nhiên, những phương pháp này lại bỏ qua các logic nghiệp vụ quan trọng của ứng dụng. Việc này dẫn đến thiếu sót các kịch bản kiểm thử quan trọng, đặc biệt là những kịch bản khó tiếp cận, mà chỉ có những chuyên gia kiểm thử với kiến thức về nghiệp vụ ứng dụng mới có thể nhận diện.

Cuối cùng, một vấn đề lớn trong kiểm thử tự động là sự thiếu hụt kiến thức từ người kiểm thử. Kinh nghiệm của người kiểm thử, đặc biệt là khi kiểm tra ứng dụng từ góc độ giao diện người dùng (GUI), là rất quan trọng. Tuy nhiên, các công cụ kiểm thử tự động hiện tại thường không thể tích hợp đầy đủ kiến thức từ người kiểm thử, dẫn đến việc bỏ sót các tình huống hoặc kịch bản mà chỉ có người kiểm thử mới có thể phát hiện và xử lý.

Chính vì những vấn đề này, bài báo **Practical, Automated Scenario-based Mobile App Testing** đã giới thiệu một công cụ có thể giải quyết hiệu quả cả ba vấn đề trên, mở ra một hướng đi mới trong việc tối ưu hóa và nâng cao chất lượng kiểm thử ứng dụng di động tự động.

II. PHƯƠNG PHÁP ĐỀ XUẤT

Công cụ được giới thiệu trong bài báo có tên ScenTest, là một giải pháp tự động hóa kiểm thử ứng dụng di động.

ScenTest sử dụng công nghệ thị giác máy tính (Computer Vision - CV) để phân tích và nhận diện hình ảnh của giao diện người dùng (GUI). ScenTest có khả năng tự động nhận dạng cấu trúc và đặc điểm của giao diện, từ đó xây dựng các kịch bản kiểm thử một cách tự động và chính xác.

Đối với việc xử lý hình ảnh, ScenTest áp dụng OCR (Nhận dạng ký tự quang học) để trích xuất văn bản và xác định vị trí của các đoạn văn bản trong hình ảnh. Điều này giúp xác định mối quan hệ giữa các văn bản và các thành phần giao diện người dùng (widget) nằm tại những vị trí cụ thể trên màn hình.

Tiếp theo, công cụ sử dụng thuật toán phát hiện cạnh Canny để nhận diện các đường viền của các widget, giúp xác định chính xác các ranh giới và hình dạng của chúng. Các đặc điểm giãn nở và xói mòn được áp dụng để tối ưu quá trình nhận diện và phân biệt các đối tượng trong giao diện, đảm bảo tính chính xác cao hơn trong việc xác định cấu trúc của từng thành phần.

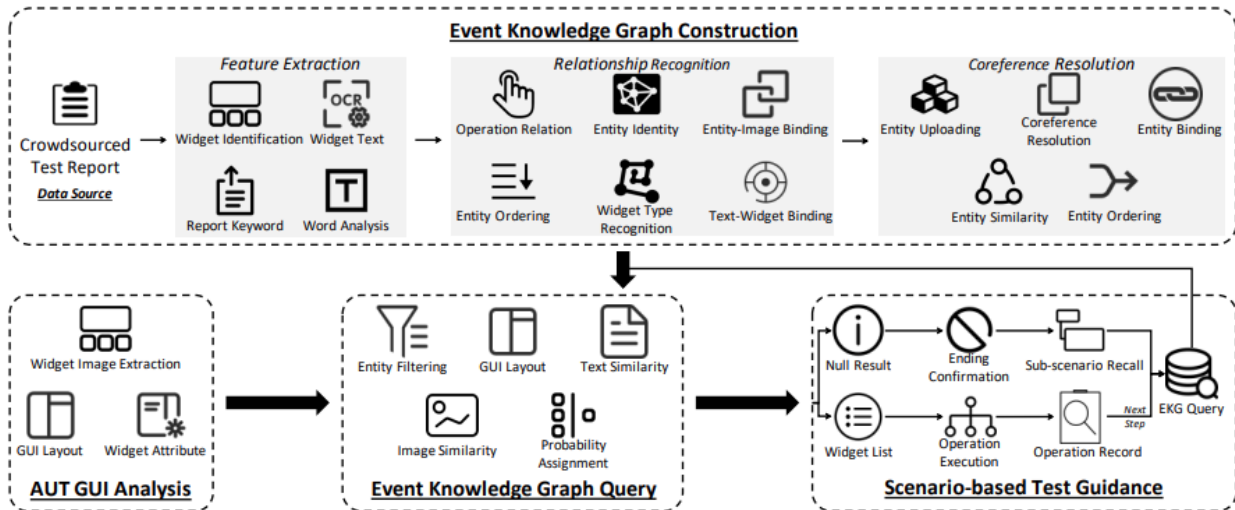
Cuối cùng, mô hình CNN (Mạng nơ-ron tích chập) được sử dụng để phân loại các widget, chẳng hạn như Button (nút bấm), TextView (trường văn bản), từ đó giúp công cụ nhận diện được loại widget và các hành động tương ứng cần kiểm thử.

Ngoài ra, ScenTest còn sử dụng thông tin văn bản được trích xuất từ các báo cáo kiểm thử cộng đồng, qua đó bổ sung và làm phong phú thêm tập kiến thức của hệ thống. Các mô tả tự nhiên bằng văn bản từ các báo cáo kiểm thử được xử lý thông qua công nghệ xử lý ngôn ngữ tự nhiên (NLP) để hiểu được nội dung và mối quan hệ giữa các thực thể.

Quá trình xử lý này sử dụng thư viện Jieba để phân tách câu thành các từ riêng biệt, loại bỏ các từ không cần thiết và làm sạch dữ liệu đầu vào. Sau đó, công cụ DDParse, một công cụ phân tích cú pháp phụ thuộc, được sử dụng để phân tích cấu trúc ngữ pháp của câu. Trong đó sử dụng các cấu trúc cần thiết và quan trọng như:

- SBV (Chủ ngữ - Động từ): Giúp nhận diện chủ thể và hành động trong câu.
- VOB (Động từ - Tân ngữ): Giúp xác định đối tượng của hành động.

Sau khi thu thập và phân tích dữ liệu, ScenTest xây dựng một đồ thị tri thức sự kiện (Event Knowledge Graph - EKG), có thể coi như một "bản đồ logic", mô tả các sự kiện, hành động và mối quan hệ giữa các thành phần trong giao diện người dùng của ứng dụng. EKG không chỉ giúp nhận diện các kịch bản kiểm thử cụ thể mà còn hướng dẫn quy trình kiểm thử ứng dụng di động tự động, kết hợp với tri thức chuyên môn về logic nghiệp vụ ứng dụng từ những người kiểm thử, mang lại hiệu quả và độ chính xác cao hơn trong việc thực hiện kiểm thử.



Hình 2: ScenTest Framework

Khi bắt đầu quá trình kiểm thử, SCENTEST chụp ảnh màn hình ứng dụng dưới dạng trạng thái hiện tại của ứng dụng. Sau đó, SCENTEST tiến hành hiểu ngữ nghĩa của ảnh chụp màn hình ứng dụng bằng cách kết hợp các thuật toán CV truyền thống và các mô hình học sâu, bao gồm nhận dạng widgets, đặc điểm bố cục GUI, trích xuất văn bản, v.v.

III. BACKGROUND

SCENTEST hỗ trợ kiểm thử tự động ứng dụng di động dựa trên kịch bản thông qua sự hướng dẫn của đồ thị tri thức sự kiện (Event Knowledge Graph - EKG). EKG được xây dựng dựa trên 4 loại thực thể và 5 mối quan hệ giữa chúng. Tuy nhiên trong quá trình xây dựng đồ thị thì có nhiều thực thể và mối quan hệ trùng nhau được trích xuất ra. Vì vậy việc giải quyết vấn đề đồng tham chiếu là rất cần thiết.

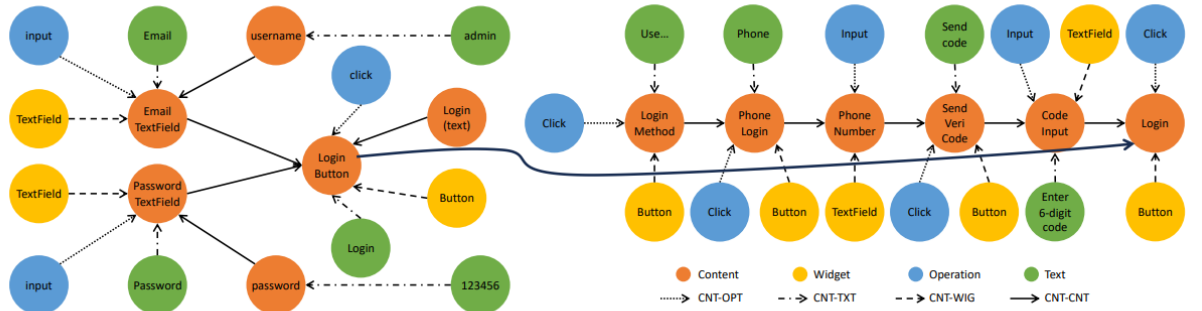
Để có thể tiếp cận phương pháp này dễ hơn thì chúng em sẽ giải thích một số khái niệm cơ bản.

1. Event Knowledge Graph - EKG

Event Knowledge Graph là biểu đồ tri thức sự kiện, nó là dạng mở rộng của đồ thị tri thức (Knowledge Graph - KG). EKG được sử dụng không chỉ để biểu diễn các thực thể và mối quan hệ giữa chúng mà nó còn thể hiện cả trạng thái cũng như là các sự kiện liên quan tới thực thể.

EKG là cây đồ thị với mỗi node trong cây đại diện cho từng thực thể được SCENTEST trích xuất. Nối giữa các node trong cây là những cạnh, mỗi cạnh lại thể hiện mối quan hệ giữa các thực thể.

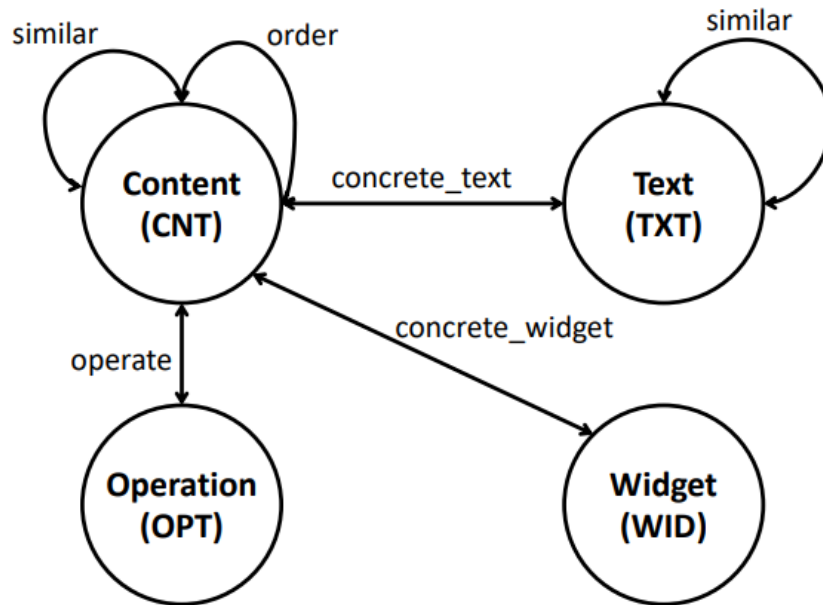
SCENTEST sử dụng EKG thay vì KG vì nó có thể biểu diễn các tri thức động là trạng thái và sự kiện thay vì chỉ đơn thuần là thực thể và các mối quan hệ tĩnh, điều này rất phù hợp trong ngữ cảnh kiểm thử.



Hình 3: Ví dụ về đồ thị EKG

2. 4 loại thực thể

Có 4 loại thực thể được SCENTEST trích xuất và sử dụng để xây dựng EKG. Phương pháp này chỉ cần tới 4 loại mà không cần nhiều hơn hay ít hơn bởi các thực thể này là những thực thể chính và nó đã bao quát được đầy đủ các yếu tố trong quá trình kiểm thử, nếu thêm nhiều thực thể khác sẽ khiến đồ thị thêm phức tạp lên nhiều mà không mang lại hiệu quả đáng kể.



Hình 4: 4 loại thực thể và 5 mối quan hệ được SCENTEST định nghĩa trong EKG

2.1. Text (TXT)

TXT bao gồm các văn bản xuất hiện trên giao diện ứng dụng, các văn bản, hoặc cũng có thể là text từ giao diện được đính kèm trong báo cáo kiểm thử. Đây là cầu nối quan trọng giúp liên kết phần tử giao diện với mô tả được viết trong báo cáo.

Ví dụ:

- Văn bản trên widget: “Login”
- Văn bản từ báo cáo kiểm thử: “Click on the Login button”

2.2. Widget (WID)

WID là các thành phần giao diện người dùng, thường là các thành phần tương tác trên ứng dụng di động, có thể coi các thực thể này là mục tiêu tác động của các thao tác.

Ví dụ: button, textbox,...

2.3. Operation (OPT)

Operation đại diện cho các hành động được thực hiện trên ứng dụng di động, thường liên quan đến một widget cụ thể, nó mô tả cách người dùng tương tác với các đối tượng.

Ví dụ: click, input, press,...

2.4. Content (CNT)

CNT đại diện cho các thực thể logic hoặc khái niệm trong ứng dụng, thường được liên kết với các widget cụ thể hoặc các văn bản mô tả hành động, cung cấp logic của các thành phần GUI. Nói một cách đơn giản, CNT là sự hợp thành của các loại thực thể là WID, TXT và OPT.

Ví dụ: Login button, Password textfield, Submit button,...

3. 5 mối quan hệ

Cùng với 4 loại thực thể, SCENTEST cũng trích xuất ra 5 mối quan hệ để thể hiện sự liên kết giữa chúng.

3.1. TXT-TXT

Giữa các thực thể TXT tồn tại mối quan hệ tương tự - đây là mối quan hệ phổ biến trong các văn bản, kết nối các văn bản tương tự hoặc đồng nghĩa. Gồm 3 trường hợp:

- Sự tương tự giữa văn bản trong GUI và mô tả trong báo cáo
- Sự tương tự giữa văn bản giữa các báo cáo
- Sự tương tự giữa văn bản trong các widget có chứa văn bản

Ví dụ: mô tả trong các văn bản có thể sử dụng “nhập” và “nhấn”, tuy 2 từ khác nhau nhưng chúng cùng thể hiện một hành động.

3.2. CNT-OPT

Mối quan hệ vận hành - liên kết giữa một thực thể logic và thao tác liên quan, thể hiện rằng hành động OPT có thể thực hiện trên đối tượng CNT.

Ví dụ: OPT “click” có thể thực hiện trên thực thể CNT “Login button”

3.3. CNT-TXT

Mối quan hệ này liên kết các văn bản cụ thể với thực thể CNT logic. Các văn bản này được trích xuất từ ảnh chụp màn hình GUI của ứng dụng hoặc văn bản trong báo cáo và sẽ làm phong phú thêm các thuộc tính của thực thể CNT logic, làm cho nó rõ ràng hơn trong quá trình truy vấn EKG.

Ví dụ: “Login button” và nhãn “Login”, Password TextField và Placeholder "Enter your password",...

3.4. CNT-WID

Mối quan hệ liên kết giữa một khái niệm logic với 1 widget cụ thể. Đối với mỗi hình ảnh đại diện cho các widget GUI, SCENTEST trích xuất các văn bản hiện có và loại widget. Sau đó, các hình ảnh có thể được liên kết với các thực thể CNT logic để tạo thành mối quan hệ CNT- WID.

Ví dụ: “Submit button” có mối quan hệ CNT-WID với loại widget là “button”.

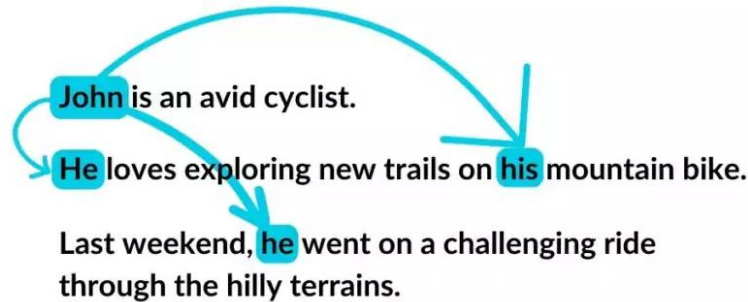
3.5. CNT-CNT

Quan hệ giữa các thực thể logic tương đồng hoặc có mối quan hệ về thứ tự. Quan hệ này được sử dụng để hỗ trợ giải quyết đồng tham chiếu và truy vấn EKG. Có 2 loại quan hệ:

- Quan hệ tương đồng: thể hiện mức độ tương đồng giữa các thực thể logic.
- Quan hệ thứ tự: chỉ ra giữa 2 thực thể logic CNT cụ thể, có thể tồn tại thứ tự thao tác. Tức là một số thực thể CNT cần được thao tác trước những thực thể khác. Quan hệ thứ tự được xác định từ các bước mô tả trong các báo cáo, được sắp xếp theo trình tự thao tác của người dùng.

4. Giải quyết đồng tham chiếu

Các thực thể được trích xuất từ các báo cáo kiểm thử cộng đồng khác nhau có thể trùng lặp do ngôn ngữ của những người trình bày báo cáo khác nhau. Ví dụ như hình dưới, “John” và các từ “he”, “his” đều dùng với mục đích chỉ 1 chủ thể chung.



Hình 5: Ví dụ đồng tham chiếu

Vì vậy giải quyết đồng tham chiếu là rất cần thiết để:

- Xác định sự dư thừa: xác định được rằng các thực thể được mô tả khác nhau nhưng thực chất là một
- Hợp nhất thông tin: kết hợp các mô tả khác nhau từ các báo cáo khác nhau để bổ sung các thông tin còn thiếu, giúp làm phong phú tri thức trong EKG.
- Giữ lại thông tin: không trực tiếp loại bỏ thông tin luôn vì mỗi mô tả khác nhau có thể phù hợp với ngữ cảnh ứng dụng hoặc kịch bản kiểm thử khác nhau.

Giải quyết đồng tham chiếu liên quan chặt chẽ đến mối quan hệ tương tự giữa các thực thể. Trong thiết kế EKG của SCENTEST, hai loại thực thể có mối quan hệ tương tự là CNT và TXT.

Cụ thể, giải quyết đồng tham chiếu dựa trên phép tính sự tương đồng hai bước. SCENTEST thực hiện tính toán độ tương đồng giữa hình ảnh GUI và văn bản. Các hình ảnh GUI có chứa văn bản, do đó chính các văn bản trong hình ảnh GUI có thể đại diện cho hình ảnh GUI.

- Bước 1: Tính toán độ tương đồng văn bản (Text Similarity Calculation)
 - SCENTEST sử dụng một bộ dữ liệu từ đồng nghĩa (synonym dataset) để so sánh sự tương đồng giữa các từ khóa vì quá trình kiểm thử có thể sử dụng nhiều từ ngữ có ý nghĩa tương tự như “press” và “click”.
 - Nếu không tìm thấy từ đồng nghĩa, SCENTEST tính toán độ tương đồng văn bản bằng cách sử dụng khoảng cách Euclid. Để có thể tính toán được thì văn bản sẽ được chuyển thành vector bằng mô hình Word2Vec. Sau đó, tính toán khoảng cách Euclid giữa các vector để xác định mức độ tương đồng giữa các văn bản. Khoảng cách vector càng nhỏ thì độ tương đồng càng lớn
- Bước 2: Tính toán độ tương đồng hình ảnh GUI (GUI Image Similarity Calculation)

- SCENTEST trích xuất các hình ảnh GUI, sử dụng văn bản trên hình ảnh GUI đó và so sánh với văn bản từ hình ảnh trên báo cáo. Nếu văn bản trùng khớp thì hình ảnh GUI cũng được coi là tương đồng.
- Với những trường hợp khó để so sánh, SCENTEST có thể sử dụng thêm các kỹ thuật xử lý hình ảnh như so sánh pixel hoặc trích xuất đặc trưng hình ảnh để xác định sự giống nhau giữa các widget (ví dụ: nút bấm có hình dạng hoặc màu sắc giống nhau).
- Các thực thể trên giao diện GUI được coi là tương đồng nếu chúng có cùng văn bản mô tả và có độ tương đồng hình ảnh cao.
- Sau khi thực hiện 2 bước trên, SCENTEST sẽ bắt đầu hợp nhất các thực thể tương tự, xây dựng mối quan hệ tương đồng giữa các thực thể và bổ sung các thông tin để làm giàu tri thức.

IV. PROCESS

1. Xây dựng cây đồ thị EKG

Algorithm 1 EKG Construction

Input: Crowdsourced test report set $\mathbb{R}$

Output: EKG $\mathbb{G}$

```

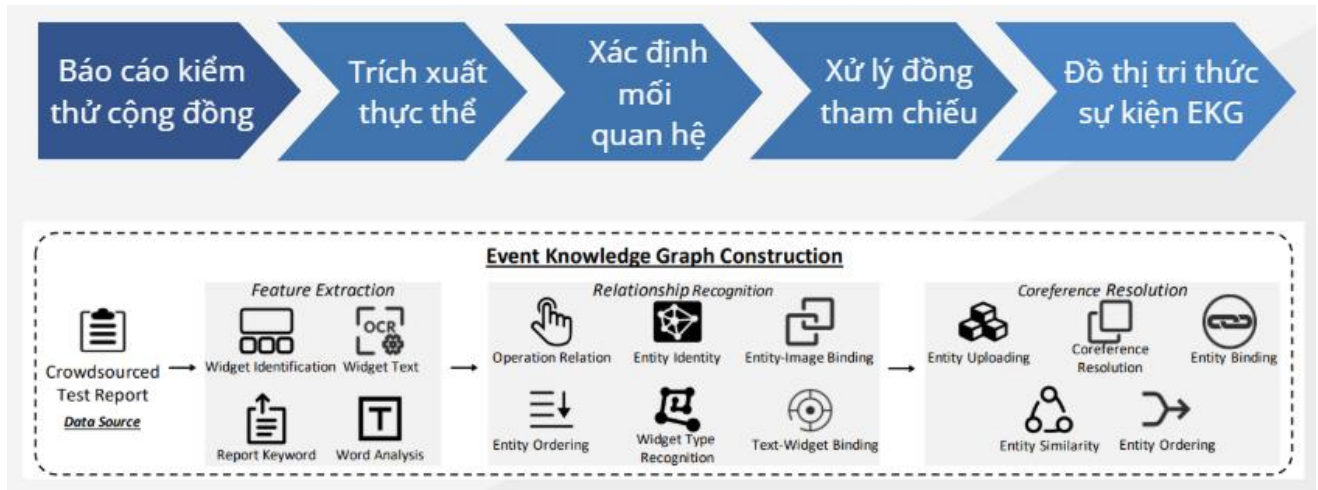
1: for  $report \in \mathbb{R}$  do
2:    $Entity_{image} \leftarrow \text{extractEntity}(report.image)$ 
3:    $Entity_{text} \leftarrow \text{extractEntity}(report.text)$ 
4:    $Entity \leftarrow Entity_{image} + Entity_{text}$ 
5:   for  $entity \in Entity$  do
6:      $type \leftarrow \text{identifyEntityType}(entity)$ 
7:      $\mathbb{G}.\text{appendEntity}(entity, type)$ 
8:   end for
9:   for  $entity_a \in Entity$  do
10:    for  $entity_b \in Entity$  do
11:       $relation \leftarrow \text{relationRecog}(entity_a, entity_b)$ 
12:       $\mathbb{G}.\text{appendRelation}(entity_a, entity_b, relation)$ 
13:    end for
14:  end for
15:  for  $entity_a \in Entity$  do
16:    for  $entity_b \in Entity$  do
17:      if  $\text{corefResol}(entity_a, entity_b) == \text{True}$  then
18:         $\text{addSimilarRelation}(entity_a, entity_b)$ 
19:         $\mathbb{G}.\text{update}()$ 
20:      end if
21:    end for
22:  end for
23: end for
24: return  $\mathbb{G}$ 

```

Hình 6: Thuật toán 1: Xây dựng đồ thị EKG

Nguồn dữ liệu để xây dựng EKG là các báo cáo kiểm thử cộng đồng. Những báo cáo này bao gồm các sự kiện kiểm thử thủ công, mô tả chi tiết các bước thao tác, các widget kèm theo là ảnh chụp màn hình ứng dụng tại từng bước.

Từ các báo cáo kiểm thử cộng đồng ban đầu, dữ liệu được phân tích và xử lý qua ba bước chính: trích xuất thực thể, xác định mối quan hệ và xử lý đồng tham chiếu, và kết quả cuối cùng là đồ thị tri thức sự kiện.

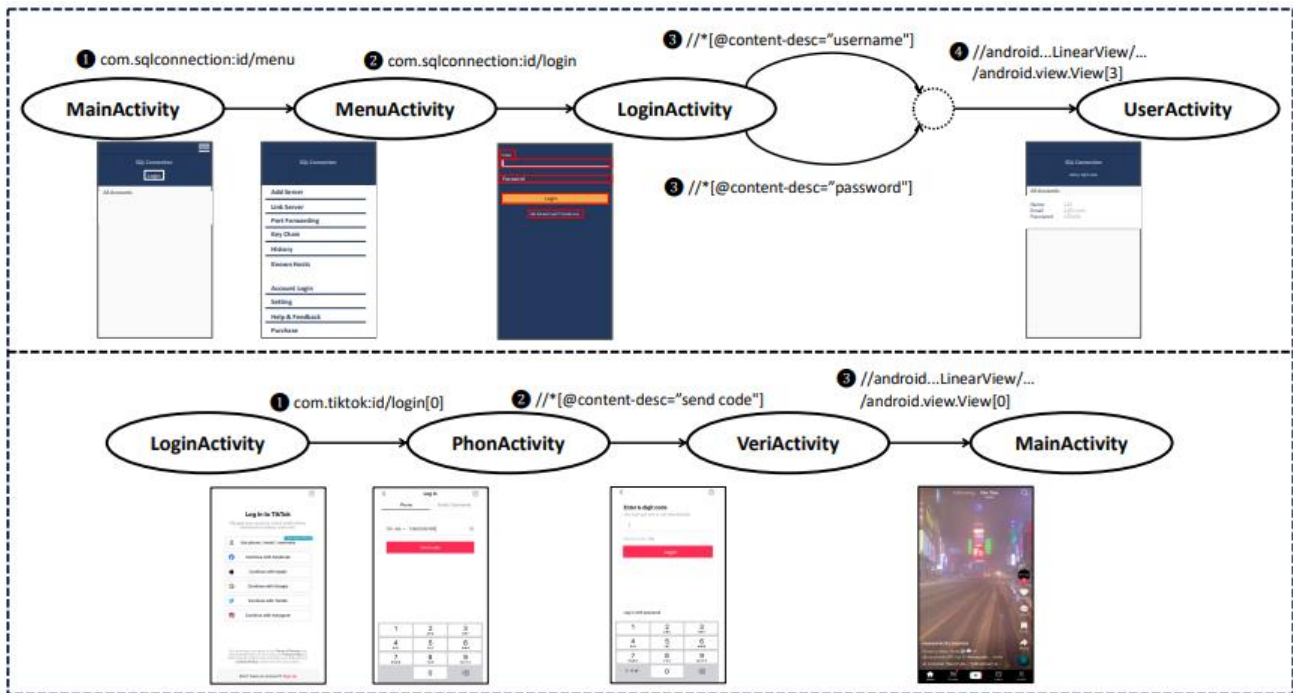


Hình 7: Process Xây dựng đồ thị EKG

1.1. Trích xuất thực thể

Quá trình trích xuất tập trung vào ảnh chụp màn hình và mô tả văn bản. Ảnh chụp màn hình ứng dụng cung cấp thông tin tĩnh về trạng thái ứng dụng, trong khi phần mô tả văn bản cung cấp thông tin động về các thao tác kiểm thử. Những thông tin này được trích xuất thành các thực thể để hỗ trợ kiểm thử theo kịch bản.

- Định danh widget (Widget Identification): Đây là quá trình xác định các phần tử giao diện người dùng trong ứng dụng, bao gồm các thành phần như nút bấm, hộp văn bản.
- Văn bản widget (Widget Text): Sử dụng OCR để phát hiện các văn bản trên ảnh chụp màn hình như nhãn, chỉ dẫn, hoặc thông báo.
- Từ khóa báo cáo (Report Keyword): Trích xuất các từ khóa quan trọng trong báo cáo kiểm thử. Ví dụ, từ khóa như "lỗi", "không phản hồi", hay "thành công".
- Phân tích từ (Word analysis): Xác định và phân tích các từ có nghĩa tương tự hoặc giống nhau trong ứng dụng.



Hình 8: Ví dụ từ báo cáo kiểm thử cộng đồng

1.2. Xác định mối quan hệ

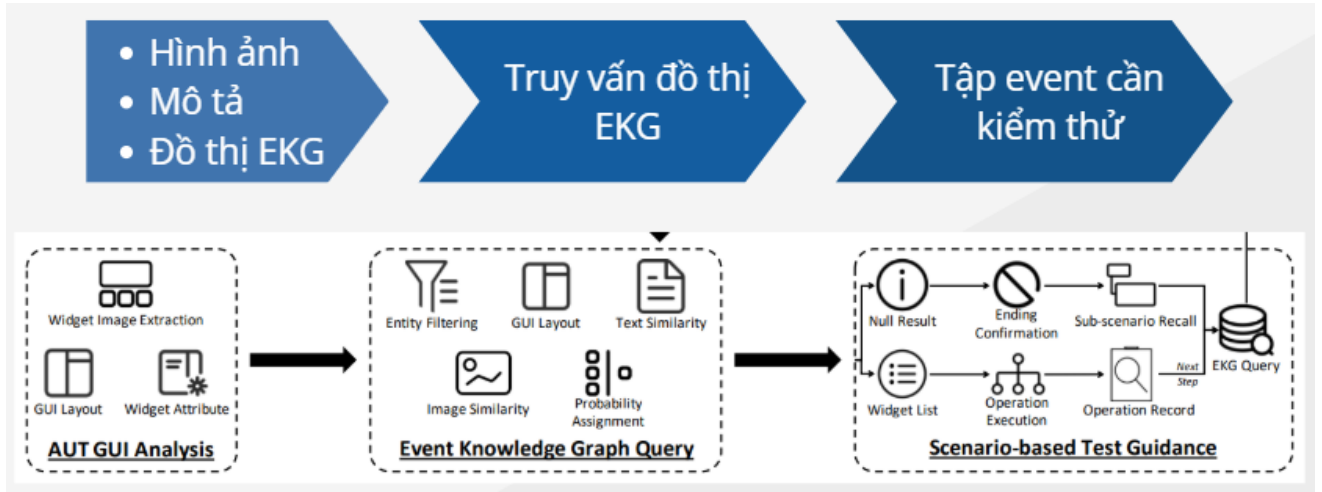
- **Mối quan hệ thao tác (Operation Relation):** Đây là quá trình nhận diện và xác định các tương tác của người dùng với ứng dụng, như việc nhấn nút hoặc nhập liệu vào các trường thông tin. Mối quan hệ này giúp nhận diện các thao tác người dùng thực hiện, từ đó liên kết chúng với các thực thể và đối tượng trong ứng dụng.
- **Định danh thực thể (Entity Identity):** Quá trình này liên quan đến việc định các thực thể trong hệ thống, bao gồm cả việc nhận diện hình ảnh và văn bản liên quan đến chúng. Ví dụ, việc gán tên hoặc mô tả cho một hình ảnh hoặc đoạn văn bản để giúp hệ thống hiểu rõ và phân biệt các thực thể trong quá trình kiểm thử.
- **Liên kết thực thể - ảnh (Entity-Image Binding):** Đây là việc kết nối các thực thể (như các widget hoặc các đối tượng giao diện) với những hình ảnh tương ứng của chúng. Ví dụ, khi có một nút "Home" trong giao diện, hình ảnh tương ứng của nút đó là icon hình ngôi nhà.
- **Thứ tự thực thể (Entity Ordering):** Mối quan hệ này xác định thứ tự và vị trí của các thực thể trong không gian giao diện người dùng. Điều này có thể bao gồm việc xác định các thực thể xuất hiện trước hoặc sau trong một chuỗi thao tác, cũng như mối quan hệ không gian giữa chúng (ví dụ, một nút nằm bên trái một trường nhập liệu).
- **Nhận dạng widget (Widget Type Recognition):** Quá trình phân loại các widget trong ứng dụng, như xác định liệu một đối tượng là nút bấm, trường nhập liệu, hay danh sách thả xuống. Việc phân loại này giúp xác định loại tương tác mà người dùng có thể thực hiện trên các đối tượng giao diện khác nhau.

- Liên kết văn bản - widget (Text-Widget Binding): Đây là việc gắn nhãn văn bản với các widget tương ứng trong ứng dụng. Ví dụ, một đoạn văn bản như "Nhập mật khẩu" sẽ được liên kết với trường nhập liệu mật khẩu. Mối quan hệ này giúp làm rõ ngữ nghĩa và chức năng của các widget trong giao diện người dùng.

1.3. Giải quyết đồng tham chiếu

- Tải lên thực thể (Entity Uploading): Đẩy lên các thực thể đã được nhận diện từ các báo cáo kiểm thử hoặc dữ liệu ban đầu để có thể xử lý, phân tích và sử dụng chúng trong các bước tiếp theo của quá trình xây dựng biểu đồ tri thức sự kiện (EKG).
- Giải quyết đồng tham chiếu (Coreference Resolution): Đây là bước liên kết các thực thể giống nhau, dù chúng có thể được mô tả bằng những từ ngữ khác nhau. Mục tiêu là đảm bảo tính nhất quán và tránh lặp lại các thực thể trong hệ thống, đồng thời kết hợp thông tin từ các mô tả khác nhau để hoàn thiện sự hiểu biết về các thực thể đó.
- Tương đồng thực thể (Entity Similarity): Đánh giá độ giống nhau của các thực thể, bao gồm việc so sánh và tính toán mức độ tương đồng giữa các thực thể dựa trên các đặc điểm như tên, mô tả, hoặc hình ảnh. Quá trình này giúp xác định các thực thể có thể được coi là tương đồng hoặc là một phần của cùng một nhóm trong EKG.
- Thứ tự thực thể (Entity Ordering): Sắp xếp các thực thể theo một trình tự logic trong biểu đồ tri thức sự kiện (EKG), đảm bảo rằng các thực thể được đặt đúng vị trí trong mối quan hệ với nhau. Điều này giúp xây dựng một quy trình kiểm thử mạch lạc, thể hiện được sự liên kết và thứ tự mà các sự kiện hoặc hành động diễn ra trong kịch bản kiểm thử.
- Liên kết thực thể (Entity Binding): Là quá trình liên kết các thực thể đã được nhận diện và xác định mối quan hệ giữa chúng. Điều này giúp tổ chức và xây dựng các thực thể thành một mạng lưới thông tin có thể được sử dụng cho các phân tích và kiểm thử tiếp theo.

2. Kiểm thử ứng dụng di động dựa trên kịch bản



Hình 9: Process Kiểm thử trên ứng dụng di động

Quá trình phân tích và trích xuất các thực thể được mô tả ở mã giả dưới:

Algorithm 2 Scenario-based Mobile App Testing

Input: App Current State $\mathcal{S}$, EKG $\mathcal{G}$

Output: Test Event $\mathbb{E}$

```

1: initialize test context  $\mathbb{C}$ 
2:  $Entity_{app} \leftarrow widgetExtraction(\mathcal{S})$ 
3: for  $entity \in Entity_{app}$  do
4:    $entity.attach(entity.attribute())$ 
5: end for
6: layoutCharacterization( $\mathcal{S}$ )
7: for  $entity_{app} \in Entity_{app}$  do
8:   for  $entity_{EKG} \in \mathcal{G}$  do
9:     if  $match(entity_{app}, entity_{EKG})$  then
10:      if  $entity_{app}.match(\mathbb{C})$  then
11:         $\mathbb{E}.add(entity_{app})$ 
12:      end if
13:    end if
14:  end for
15: end for
16: return  $\mathbb{E}$ 

```

Hình 10: Thuật toán Kiểm thử ứng dụng dựa trên kịch bản

- Đầu vào: trạng thái hiện tại của ứng dụng, đồ thị tri thức sự kiện
- Đầu ra: tập các sự kiện kiểm thử ứng dụng
- S: Trạng thái hiện tại của ứng dụng (ảnh chụp giao diện GUI)
- G: đồ thị EKG lưu trữ các thực thể và các mối quan hệ
- E: Danh sách các sự kiện kiểm thử được lấy từ ứng dụng và EKG, ban đầu là rỗng

- C: ngữ cảnh kiểm thử, chứa thông tin về các hành động đã thực hiện trước đó trong quá trình kiểm thử

SCENTEST sử dụng thuật toán widgetExtraction() để trích xuất các widget từ trạng thái giao diện, mỗi widget được coi là một thực thể. Với mỗi thực thể được trích xuất từ GUI ứng dụng thì thuật toán sẽ trích xuất các thuộc tính của nó như vị trí, văn bản, loại,... sau đó gắn các thuộc tính này vào widget tương ứng thông qua phương thức attach()

Phương thức layoutCharacterization(S) phân tích bố cục giao diện để xác định cách các widget được thiết kế, quan hệ không gian giữa các widget. Điều này giúp SCENTEST hiểu được ngữ cảnh giao diện.

Sau đó, với mỗi thực thể được trích xuất từ GUI ứng dụng sẽ được so khớp với từng thực thể trong đồ thị EKG, nếu thực thể trong đồ thị tương đồng với thực thể được trích xuất từ GUI ứng dụng thì nó sẽ được so với ngữ cảnh kiểm thử hiện tại, nếu phù hợp thì nó sẽ thêm vào tập sự kiện kiểm thử E. Quá trình đó cứ lặp lại cho đến khi xét qua hết các thực thể được trích xuất từ GUI ứng dụng.

Kết quả của thuật toán là danh sách E chứa tất cả các sự kiện kiểm thử.

2.1. Phân tích GUI ứng dụng

Phân tích GUI ứng dụng nhằm vào trạng thái hiện tại của ứng dụng. SCENTEST hiểu biết ngữ cảnh ứng dụng, trích xuất các tiện ích, thu thập thuộc tính của tiện ích và các đặc điểm bố cục GUI.

Đầu tiên, SCENTEST áp dụng thuật toán OCR để trích xuất tất cả các văn bản hiện có từ ảnh chụp màn hình GUI của ứng dụng. Sau đó sử dụng thuật toán phát hiện cạnh là Canny để trích xuất các đường viền widget. Ngoài ra, SCENTEST còn kết hợp sử dụng CNN để xác định các loại widget (như button, textfield,...) để hỗ trợ tốt hơn trong việc nhận dạng mối quan hệ và các truy vấn EKG.

❖ Trích xuất thành phần giao diện

Quá trình này sử dụng thuật toán phát hiện cạnh là Canny để trích xuất các đường viền widget kết hợp với các biện pháp làm mờ hoặc làm đậm để trích xuất được chính xác hơn.

Ví dụ: thuật toán phát hiện cạnh (edge detection) phát hiện các đường viền xung quanh nút bấm và textfield dựa trên sự tương phản màu sắc/độ sáng

Dilation và Erosion: phóng to để làm rõ đường viền của các thành phần như nút bấm và xóa bỏ các chi tiết nhỏ hoặc các họa tiết không quan trọng

❖ Lấy thuộc tính của thành phần

Bên cạnh hình ảnh của các widget thì chúng cũng còn nhiều thông tin ngữ nghĩa hơn có thể thu thập.

Đầu tiên là tọa độ widget, chúng cũng rất quan trọng vì nhờ chúng mà có thể phản ánh mối quan hệ giữa tất cả các widget.

Thứ hai là loại widget. Các loại widget không thể suy ra đơn giản mà cần được nhận dạng bằng mô hình nơ-ron tích chập CNN. Loại widget có thể được gắn với các thao tác cụ thể.

Thứ ba là các văn bản trên widget được nhận dạng bằng thuật toán OCR (nhận dạng ký tự quang học). SCENTEST không trích xuất văn bản từ bố cục GUI vì:

- Bố cục GUI có thể thiếu thông tin trên một số thành phần đặc biệt (ví dụ: "canvas" widget - không có nội dung bên trong).
- Một số nội dung từ các trang HTML nhúng cũng không thể trích xuất được.

Sau khi trích xuất, các thuộc tính ngữ nghĩa (tọa độ, loại, văn bản) sẽ được gắn liền với từng thành phần tương ứng. Điều này giúp dễ dàng liên kết các thành phần với các thực thể trong EKG (Đồ thị tri thức mở rộng - Extended Knowledge Graph)

❖ **Đặc trưng hóa bố cục GUI**

Các thành phần giao diện đã được trích xuất cùng với thuộc tính của chúng sẽ được gửi dưới dạng một truy vấn tới EKG để phân loại và ghép nối thực thể.

Bước đặc trưng hóa bố cục dựa vào tọa độ của các thành phần giao diện.

- Tọa độ x, y: Dựa vào tọa độ (x,y) để xác định các thành phần có cùng hàng, cùng cột
- Cấu trúc cây: Các thành phần giao diện sẽ được tổ chức theo dạng cây, trong đó mỗi nút cây là một thành phần giao diện (widget), và các thuộc tính sẽ được gắn với các nút này.
- Xử lý các thành phần gần nhau: Đối với các thành phần có tọa độ dọc hoặc ngang chỉ khác nhau một chút (khoảng 0.1 kích thước ảnh chụp màn hình) sẽ được áp dụng để gộp các tọa độ gần nhau lại, giúp giảm bớt các phân cấp

2.2. Sinh ca kiểm thử dựa trên kịch bản

Sự hướng dẫn của EKG cho quá trình kiểm thử chủ yếu dựa trên 2 nguồn thông tin: trạng thái ứng dụng hiện tại (thông tin GUI) và bối cảnh kiểm thử. Bối cảnh kiểm thử có ảnh hưởng rất lớn đến việc kiểm thử bởi nó sẽ ảnh hưởng tới sự xảy ra của các sự kiện kiểm thử.

Quá trình chuyển đổi sự kiện của ứng dụng di động có thể coi như một đồ thị có hướng. Kịch bản phụ (sub-scenario) có thể được xem như 1 nhánh trong đồ thị này, đi từ một nút này sang một nút đuôi khác và từ nút bắt đầu đó có thể có nhiều đường dẫn khác nhau có chung 1 mục tiêu (nút đuôi). Nói một cách khái quát, kịch bản phụ nghĩa là một đường dẫn độc lập trong đồ thị có thể thực thi theo và hoàn thành kịch bản kiểm thử, nó như một khung hướng dẫn SCENTEST thực hiện các sự kiện trong quá trình kiểm thử. Một kịch bản phụ có thể sử dụng nhiều đầu vào khác nhau để thực thi. Nếu đầu vào hợp lệ thì việc kiểm thử sẽ dễ dàng, nhưng nếu không hợp lệ thì sẽ bị dừng giữa chừng do lỗi của các kịch bản trong đồ thị. Vì vậy, SCENTEST sử dụng đầu vào là các dữ liệu đã được sử dụng cho kiểm thử được lấy từ các báo cáo cộng đồng.

Ví dụ: với kịch bản “Đăng nhập” có thể có 2 kịch bản phụ như: “người dùng có thể đăng nhập với tên và mật khẩu” hoặc “người dùng có thể đăng nhập với số điện thoại và mã xác minh”

Quá trình tạo kiểm thử có thể được hiểu như quá trình khám phá theo chiều sâu, mỗi khi kịch bản phụ kết thúc (đi tới điểm đuôi) thì trạng thái ứng dụng lại được gọi lại thời điểm mà bắt đầu điểm nhánh và thực thi trên nhánh tiếp theo cho tới khi tất cả các nhánh đều được kiểm thử.

Để có được các bước thực hiện kiểm thử, SCENTEST sẽ gửi truy vấn chứa trạng thái cùng với ngữ cảnh hiện tại của ứng dụng tới EKG. Trạng thái hiện tại của ứng dụng sẽ được phân tích thành một tệp cấu trúc GUI có dạng lồng ghép. Khi truy vấn tới EKG, tất cả các phần tử widget trong tệp cấu trúc GUI sẽ được so khớp với từng nút trong EKG để tìm ra widget nào sẽ được thực thi. Để tìm được một widget thực thi thì cần xác định dựa trên 2 tiêu chí: widget trên trạng thái GUI hiện tại phải nằm trên một kịch bản phụ phù hợp với bối cảnh kiểm thử hiện tại và sự tương đồng giữa thực thể trên GUI với EKG. Xác suất một widget được chọn để thực hiện được định nghĩa trong Phương trình bên dưới:

$$P = \begin{cases} 0, & w \notin context \\ similarity(w, e), & w \in context \end{cases} \quad (1)$$

Hình 11: Công thức xác suất một widget được chọn

Trong đó:

- w đại diện cho widget được trích xuất từ ứng dụng GUI
- e đại diện cho các thực thể trong EKG

Đối với bước đầu tiên, SCENTEST kiểm tra widget có nằm trong một nhánh kịch bản nào thích hợp hay không. Các nút trạng thái trong EKG được khớp với các nút GUI hiện tại của ứng dụng từ đó thể hiện được trạng thái ứng dụng đã được kiểm thử những gì. Nếu các nút khớp nhau thì widget được cho là nằm trong đường dẫn kịch bản thích hợp. Nếu các nút không nằm trong kịch bản thích hợp thì nó sẽ không được thực thi bằng cách thiết lập xác suất là 0. Điểm cần lưu ý trong bước này là các widget được chọn để so khớp là các thực thể được đánh dấu là điểm bắt đầu (có nhãn starting point)

Với bước thứ hai, SCENTEST so sánh các widget trong trạng thái hiện tại của ứng dụng với các thực thể trong EKG để tính mức độ tương đồng, widget có mức độ tương đồng cao nhất sẽ được chọn. Cuối cùng, SCENTEST sẽ tương tác với một và chỉ một widget được chọn đó và nó sẽ được thêm vào bối cảnh kiểm thử sau khi tương tác. Nếu widget được gắn với thẻ tail đánh dấu một kịch bản phụ kết thúc.

V. RESEARCH QUESTION - THỰC NGHIỆM BÀI BÁO

Bài báo đưa ra một phương pháp tiếp cận mới là kết hợp kiến thức chuyên môn của người kiểm thử vào kiểm thử ứng dụng di động dựa trên kịch bản. Do đó, nó sẽ khác so với các phương pháp tiếp cận kiểm thử tự động truyền thống như code coverage hay sinh ngẫu nhiên... Vì vậy không thể đánh giá công cụ theo các phương pháp truyền thống được.

Các tác giả đã đặt ra năm câu hỏi nghiên cứu (Research Question) từ các khía cạnh khác nhau để đánh giá công cụ SCENTEST một cách toàn diện. Đó là:

- RQ1 (Tính đúng đắn): SCENTEST có thể xây dựng biểu đồ tri thức sự kiện chính xác cho kiểm thử dựa trên kịch bản không?
- RQ2 (Độ tin cậy): EKG của SCENTEST có thể đưa ra các dự đoán đáng tin cậy cho kiểm thử dựa trên kịch bản?
- RQ3 (Độ chính xác): SCENTEST có thể kiểm tra tất cả các kịch bản trên các ứng dụng khác nhau dựa trên EKG với độ chính xác như thế nào?
- RQ4 (Tính đầy đủ): SCENTEST có thể bao phủ đầy đủ các nhánh kịch bản trên các ứng dụng khác nhau không?
- RQ5 (Tính hữu ích): Liệu SCENTEST có thể phát hiện lỗi một cách hiệu quả cho các kịch bản cụ thể?

Trước hết, chúng ta cần phải xác định các đối tượng cũng như phạm vi thử nghiệm và nguồn dữ liệu báo cáo cộng đồng.

1. Đối tượng thử nghiệm

Bài báo dựa trên 2 tiêu chí để lựa chọn các kịch bản kiểm thử và kịch bản phải đạt ít nhất 1 tiêu chí đánh giá. Thứ nhất kịch bản phải được sử dụng rộng rãi trong các ứng dụng thuộc nhiều mục khác nhau. Và thứ hai, Kịch bản phải được phổ biến rộng rãi ở các loại ứng dụng cụ thể thông dụng và phổ biến với người dùng. Từ đó, rút ra được 8 kịch bản đó là Đăng nhập, Đăng ký, Email, Vé máy bay, AddCart, Chat, Âm nhạc và Video.

Về báo cáo kiểm thử từ cộng đồng, tác giả sử dụng một tập dữ liệu chứa 22.720 báo cáo kiểm thử từ 70 ứng dụng di động thực tế có mức độ phổ biến lớn để xây dựng biểu đồ tri thức sự kiện. Các báo cáo này được thu thập từ 1 nền tảng kiểm thử cộng đồng được sử dụng rộng rãi, MoocTest - một trong số các nền tảng kiểm thử cộng đồng tiêu biểu tại Trung Quốc, gần như giống hệt với các nền tảng thương mại quy mô lớn, như Global App Testing, Baidu Crowdfunding, Testin,...

Trong quá trình chứng minh thử nghiệm, có 124 ứng dụng khác nhau được sử dụng để đánh giá hiệu quả trong RQ1 đến RQ4. Đối với RQ5, có sử dụng một tập con gồm 124 ứng dụng, tất cả đều là open-source, do chúng cần được gắn công cụ để thu thập log. Trong bảng 1, có thể thấy tổng cộng có 945 event cùng 185 sub-scenario khác nhau.

Bảng 1: Thống kê số lượng ứng dụng, event, scenario kiểm thử

APPS IN THE EMPIRICAL EVALUATION

Scenario	EKG Construction	Testing	# Sub- Scenario	Event
Login	10	30	37	245
Register	10	30	59	343
Email	7	17	10	71
FlightTicket	8	19	11	49
AddCart	10	21	10	53
Chat	7	16	15	47
Music	8	19	20	66
Video	10	21	23	71
Sum (with repetition)	70	124	185	945

Sau khi xây dựng EKG, chúng ta có thể thấy tổng số thực thể và số quan hệ cho 8 kịch bản kiểm thử và số lượng tương ứng của 4 thực thể và 5 mối quan hệ tương ứng là 1036 thực thể và 4749 mối quan hệ.

Bảng 2: Thống kê các Entity và Relationship trong đồ thị EKG

EVENT KNOWLEDGE GRAPH ONTOLOGY

Scenario	Entity	Content	Widget	Operation	Text	Relationship	TXT-TXT	CNT-OPT	CNT-TXT	CNT-WID	CNT-CNT
Login	300	127	17	7	149	2481	722	190	595	470	504
Register	231	88	17	5	121	1002	150	88	157	163	444
Email	73	13	17	3	40	188	10	23	75	44	36
FlightTicket	73	16	17	4	36	233	86	22	50	37	38
AddCart	82	15	17	3	47	206	42	15	57	50	42
Chat	90	18	17	4	51	221	51	19	60	40	51
Music	89	21	17	5	46	226	54	21	62	36	53
Video	98	23	17	3	55	192	28	31	57	37	39
Sum	1036	321	136	34	545	4749	1143	409	1113	877	1207

Sau đây, chúng em sẽ đi vào chi tiết để chứng minh các câu hỏi nghiên cứu này của bài báo để đánh giá SCENTEST toàn diện nhất.

2. RQ1: Tính đúng đắn

Tính đúng đắn ở đây được hiểu là liệu SCENTEST có thể xác định tốt các thao tác và các đối tượng tương ứng hay không. Bốn loại thực thể ở dạng hình ảnh từ ảnh chụp màn hình ứng

dụng test và văn bản báo cáo từ cả mô tả văn bản và ảnh chụp trạng thái ứng dụng. Nên sẽ cần đánh giá ở phần văn bản và hình ảnh.

Trong quá trình đánh giá thủ công, các chuyên gia được yêu cầu xác định tất cả các thao tác và đối tượng tương ứng rồi xác nhận chéo các kết quả để đạt được sự nhất trí. Kết quả được hiển thị ở hai cột đầu trong BẢNG 3. Chúng ta có thể thấy rằng tỷ lệ thành công của việc trích xuất tổng thể là 99,05% (936/945 100). Hơn nữa, độ chính xác trung bình và giá trị thu hồi của tám kịch bản kiểm thử đạt 96,98% và 99,02%, cho thấy SCENTEST không chỉ có thể trích xuất hiệu quả các thực thể, mà còn có rất ít các thực thể được nhận dạng sai trong quá trình trích xuất.

Bảng 3: Kết quả của RQ1 – Tính đúng đắn

EXPERIMENT RESULTS FOR RQ1: CORRECTNESS

Scenario	Test Event	# Succ	Precision	Recall	Text Match	Layout Match	% Succ
Login	245	242	99.18%	98.78%	204	29	95.10%
Register	343	340	99.71%	99.13%	287	52	98.83%
Email	71	71	94.67%	100.00%	39	32	100.00%
FlightTicket	49	49	94.23%	100.00%	17	28	91.84%
AddCart	53	53	98.15%	100.00%	21	31	98.11%
Chat	47	45	95.74%	95.74%	28	17	95.74%
Music	66	65	95.59%	98.48%	34	29	95.45%
Video	71	71	98.61%	100.00%	38	30	95.77%
Sum / Avg	945	936	96.98%	99.02%	668	248	96.36%

Thứ hai, việc khớp văn bản hình ảnh cũng rất quan trọng đối với việc nhận dạng mối quan hệ và giải quyết đồng tham chiếu. Chúng ta có thể thấy rằng đối với 8 kịch bản kiểm thử, tỷ lệ thành công trung bình tỷ lệ đạt 96,36%.

Tuy nhiên, còn tồn tại các sai sót với các nguyên nhân như: phân đoạn từ sai và bỏ sót tính từ. Ví dụ, một báo cáo nói rằng "nhấp vào nút màu xanh lá cây", trong khi đối tượng được trích xuất chỉ là "button", và có nhiều button trên ảnh chụp màn hình, vì vậy kết quả có thể dẫn đến bị hiểu sai khiến cho scenario thất bại. Lỗi trong quá trình trích xuất văn bản, lỗi trong báo cáo không khớp với ảnh chụp màn hình GUI cũng như có các widget khó xác định cũng dẫn đến những bỏ sót cho công cụ.

3. RQ2: Độ tin cậy

Để đánh giá độ tin cậy của kết quả dự đoán của EKG, tác giả đã ghi lại tất cả các ảnh chụp màn hình GUI trên các ứng dụng và dán nhãn tất cả các widget trên ảnh chụp màn hình. Sau đó, đối với mỗi ảnh chụp màn hình GUI, lấy nó làm trạng thái hiện tại đưa vào EKG và kết quả dự đoán đầu tiên của danh sách truy vấn EKG sau đó được khớp với dán nhãn thật sự.

Bảng 4: Kết quả của RQ2 – Độ tin cậy

EXPERIMENT RESULTS FOR RQ2: RELIABILITY

Scenario	Test Event	# Succ	% Succ
Login	245	242	98.78%
Register	343	318	92.71%
Email	71	67	94.37%
FlightTicket	49	46	93.88%
AddCart	53	50	94.34%
Chat	47	44	93.62%
Music	66	63	95.45%
Video	71	67	94.37%
Sum / Avg	945	897	94.69%

Từ Bảng 4, ta có thể thấy rằng tỷ lệ thành công trung bình của việc dự đoán widget là 94,69%, dao động từ 92,71% đến 98,78% giữa các kịch bản khác nhau. Tỷ lệ thành công trung bình là tỷ số giữa số lượng kết quả dự đoán thành công và tổng số dự đoán của EKG. Kết quả cho thấy SCENTEST có thể xác định các widget mục tiêu một cách hiệu quả và đáng tin cậy từ trạng thái hiện tại của ứng dụng.

4. RQ3: Độ chính xác

Với các EKG được xây dựng tự động, điều quan trọng để xem có mang lại ích lợi gì không là độ chính xác có tạo ra các step và kiểm tra các kịch bản cụ thể hay không. Ở đây đặt ra một số liệu **GenRate** để đánh giá liệu SCENTEST có thể tạo ra các ca kiểm thử hiệu quả không, được tính như sau:

$$GenRate = \frac{\# TestOp}{\# ScenOp} \times 100\% \quad (2)$$

Hình 12: Công thức đánh giá độ chính xác GenRate

Trong đó **TestOp** là các thao tác thực tế được tạo ra, và **ScenOp** là các thao tác từ EKG. Hay có thể hiểu, ScenOp đề cập đến các thao tác trong EKG có thể hoàn thành mục tiêu bắt buộc hay không.

Nếu widget mục tiêu, thao tác dự kiến và tham số tương ứng của TestOp và ScenOp hoàn toàn giống nhau, chúng ta xem đó là kiểm thử thành công. Nếu một sự kiện kiểm thử được tạo ra sai, tất cả các sự kiện kiểm tra được tạo ra tiếp theo của nó được coi là thất bại.

Kết quả có thể được nhìn thấy từ BẢNG 5, GenRate trung bình đạt 87,74%, đó là tỷ lệ thành công cao. SCENTEST đạt được kết quả tốt nhất dẫn đến kịch bản FlightTicket là 92,31% và hiệu suất trên các kịch bản Đăng nhập và Đăng ký tương đối thấp, gần 85%.

Lý do là vì hai kịch bản này phần lớn có sẵn trên nhiều ứng dụng khác nhau, có các kịch bản tương đối phức tạp. Chúng có thể chứa nhiều thiết kế liên quan đến logic nghiệp vụ riêng.

Bảng 5: Kết quả của RQ3 – Độ chính xác

EXPERIMENT RESULTS FOR RQ3: ACCURACY

Scenario	# ScenOp	# TestOp	GenRate
Login	192	163	84.90%
Register	161	137	85.09%
Email	69	61	88.41%
FlightTicket	39	36	92.31%
AddCart	85	78	91.76%
Chat	78	69	88.46%
Music	82	73	89.02%
Video	72	63	87.50%
Sum / Avg	778	680	88.43%

Bên cạnh đó, nguyên nhân thất bại xảy ra phần lớn là do các sự kiện ngẫu nhiên và không thể kiểm soát được gây ra như các cuộc gọi đến, xác minh không phải bằng robot, chuyển đổi ứng dụng bên ngoài để yêu cầu để ủy quyền ứng dụng của bên thứ ba...

5. RQ4: Tính đầy đủ

Tính đầy đủ cũng là một chỉ số quan trọng về hiệu quả của kiểm thử dựa trên kịch bản. Để đánh giá tính đầy đủ, phạm vi phủ là một phần quan trọng. Các phương pháp automation truyền thống thường sử dụng code coverage, trong khi cách tiếp cận này, sẽ dùng kịch bản phủ

như đối tượng bao phủ. Các kịch bản phụ khác nhau sẽ bao phủ được toàn bộ kịch bản kiểm thử.

Một số liệu được gọi là ScenCov được sử dụng và tính như sau:

$$ScenCov = \frac{|S_t|}{|Scen|} \times 100\%$$

Hình 13: Công thức đánh giá tính đầy đủ ScenCov

Trong đó ***St*** đề cập đến tập hợp các kịch bản phụ được SCENTEST tự động khám phá và ***Scen*** đề cập đến tập hợp tất cả các kịch bản phụ của một kịch bản kiểm thử cụ thể.

Bảng 6: Kết quả của RQ4 – Tính đầy đủ

EXPERIMENT RESULTS FOR RQ4: ADEQUACY

Scenario	# Sub-Scenario	# Covered Scenario	ScenCov
Login	67	55	82.09%
Register	39	33	84.62%
Email	24	20	83.33%
FlightTicket	8	8	100.00%
AddCart	15	12	80.00%
Chat	31	27	87.10%
Music	29	26	89.66%
Video	36	33	91.67%
Sum / Avg	249	214	87.31%

Kết quả được ghi lại trong BẢNG 6. Giá trị trung bình ScenCov của 8 kịch bản kiểm thử đạt 87,31% và SCENTEST có hiệu suất tốt nhất trên FlightTicket kịch bản là 100%. Điều này cho thấy SCENTEST có thể kiểm tra hiệu quả các kịch bản phụ đầy đủ. Nghĩa là SCENTEST có thể thực hiện hiệu quả tất cả các kịch bản phụ trong kịch bản cụ thể không.

6. RQ5: Tính hữu ích

Tính hữu ích có nghĩa là liệu SCENTEST có thể tìm lỗi hiệu quả cho các kịch bản cụ thể không. Để so sánh, bài báo sử dụng hai công cụ đại diện là Monkey và Stocat rồi cho chạy hai giờ trên mỗi ứng dụng để thu thập các lỗi. Đối với SCENTEST, không giới hạn thời gian vì nó sẽ dừng lại khi quá trình lập hoàn tất và sẽ thu thập dữ liệu về chi phí thời gian.

Trung bình, SCENTEST có thể hoàn thành kiểm thử dựa trên kịch bản trên ứng dụng trong 378,29 giây, dao động từ 152 giây đến 993 giây tùy thuộc vào độ phức tạp của EKG, số lượng kịch bản phụ, độ phức tạp GUI của Ứng dụng, v.v.

Một điều cần lưu ý là phạm vi khám phá của Monkey and Stoat là toàn bộ ứng dụng, vì vậy nó có thể tìm thấy nhiều hơn lỗi ngoài kịch bản kiểm thử mục tiêu. Trong BẢNG 7, $St - Sc$ (sce) đề cập đến số lượng bugs được phát hiện bởi Monkey/Stoat nhưng không được phát hiện bởi SCENTEST. $St \cap Sc$ đề cập đến số lượng lỗi được phát hiện cả hai bởi Monkey/Stoat và SCENTEST. $Sc - St$ đề cập đến số lượng lỗi được SCENTEST phát hiện nhưng không được phát hiện bởi Monkey/Stoat. Kết quả cho thấy SCENTEST có thể tìm thấy 158 và 172 bugs khác biệt so với Monkey and Stoat, tương ứng. Con số chiếm 53,56% và 58,31% tổng số lỗi được phát hiện của các kịch bản kiểm thử cụ thể. Ngược lại, Monkey and Stoat chỉ có thể tìm thấy 6 và 5 lỗi riêng biệt (5 lỗi do Stoat phát hiện được bao phủ bởi Monkey).

Bảng 7: Kết quả của RQ5 – Tính hữu ích

EXPERIMENT RESULTS FOR RQ5: USEFULNESS

Scenario	# App	M - Sc (all)	M - Sc (sce)	$M \cap Sc$	$Sc - M$	Total	$St - Sc$ (all)	$St - Sc$ (sce)	$St \cap Sc$	$Sc - St$	Total
Login	15	329	0	42	33	75	343	0	42	33	75
Register	15	287	2	20	18	40	301	2	19	18	40
Email	12	143	0	13	15	28	152	0	11	17	28
FlightTicket	7	142	0	3	9	12	151	0	3	9	12
AddCart	9	128	0	12	10	22	135	0	11	11	22
Chat	6	133	2	15	18	35	143	2	14	20	35
Music	7	129	1	14	34	49	144	1	12	36	49
Video	6	102	1	12	21	34	124	0	6	28	34
Sum	77	1393	6	131	158	295	1493	5	118	172	295

Tóm lại, SCENTEST có thể tìm thấy **các lỗi riêng biệt một cách hiệu quả khi được tích hợp của miền tri thức từ những người kiểm thử**. SCENTEST như một sự bổ sung mạnh mẽ cho các công cụ kiểm thử tự động truyền thống. Đặc biệt là trong việc tìm lỗi trong các kịch bản quan trọng và phổ biến trên các ứng dụng khác nhau.

VI. CÀI ĐẶT VÀ DEMO

Sau quá trình tìm hiểu và nghiên cứu, nhóm em đã cài đặt và sử dụng thành công SCENTEST với nguồn dữ liệu được lấy từ báo cáo cộng đồng có sẵn trong mã nguồn của bài báo (<https://zenodo.org/records/11118420>)

1. Cài đặt

Để sử dụng được công cụ SCENTEST, trước hết cần config các packages hỗ trợ.

- Cài đặt Python version 3.6 – 3.7
- Xem tệp **requirements.txt** cho các gói cần thiết khác. Chạy lệnh **pip install -r requirements.txt** trong folder môi trường code để cài đặt các gói cần thiết.

```
requirements.txt
1  baidu-aip==2.2.18.0
2  certifi==2021.5.30
3  chardet==4.0.0
4  idna==2.10
5  numpy==1.24.3
6  opencv-contrib-python
7  pandas==2.0.3
8  python-dateutil==2.8.1
9  pytz==2021.1
10 requests==2.25.1
11 six==1.16.0
12 urllib3==1.26.5
13 transformers==4.46.3
14 translate==3.6.1
15 # -i https://pypi.tuna.tsinghua.edu.cn/simple
```

Hình 14: File chứa các package cần cài đặt – requirements.txt

- Config môi trường liên quan để xử lý ảnh theo **/PicAnalysis/README.md**.
- Tải xuống và cấu hình neo4j, phiên bản được sử dụng khi xây dựng mã là 3.5.5. Các phiên bản hỗ trợ 3.x.x

(https://go.neo4j.com/download-thanks.html?edition=community&release=3.5.26&flavour=winzip&ga=2.178497122.752138628.1685414453-2036529335.1685414452&ql=1*uiadon*gcl au*MzA5ODQ2OTQuMTczMjQ2NDMyNg..* ga *MTkzNDk5OTQ0OS4xNzMyNDY0MzI2* ga DL38Q8KGQC*MTczMjQ2NDMyNC4xLjEuMTczMjQ2NzAwN)

Tiếp đó cần set biến môi trường cho Neo4J dẫn đến thư mục bin. Sau đó gõ lệnh để download service liên quan **neo4j.bat install-service**

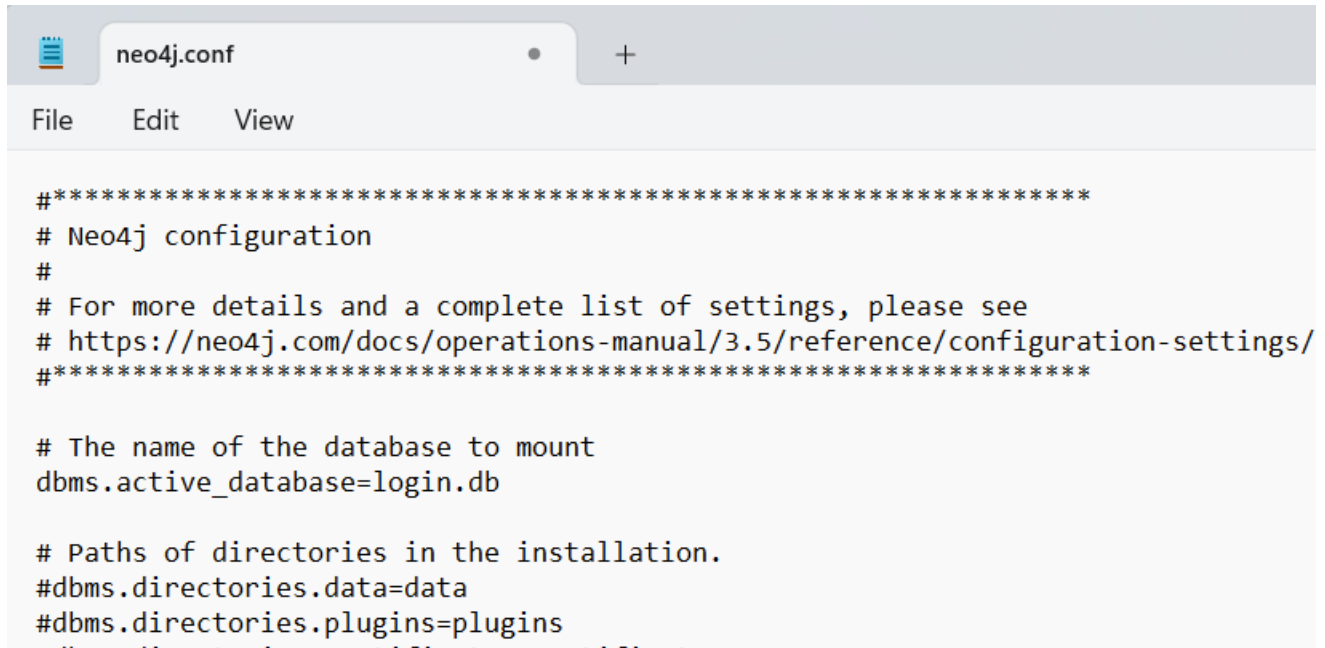
```
D:\INT3307\neo4j-community-3.5.26\bin
```

2. Cấu trúc và sử dụng đồ thị EKG

2.1. Neo4j

Đây là một cơ sở dữ liệu đồ thị, cũng có thể hiểu nó như một đồ thị tương ứng với một cơ sở dữ liệu, vì vậy đối với các chức năng khác nhau, nên thiết lập các cơ sở dữ liệu đồ thị khác nhau. Ví dụ: chúng ta tạo một cơ sở dữ liệu đồ thị có tên login để lưu trữ tất cả dữ liệu liên quan đến chức năng đăng nhập và tạo một cơ sở dữ liệu đồ thị có tên airport để lưu trữ tất cả dữ liệu liên quan đến việc đặt vé máy bay.

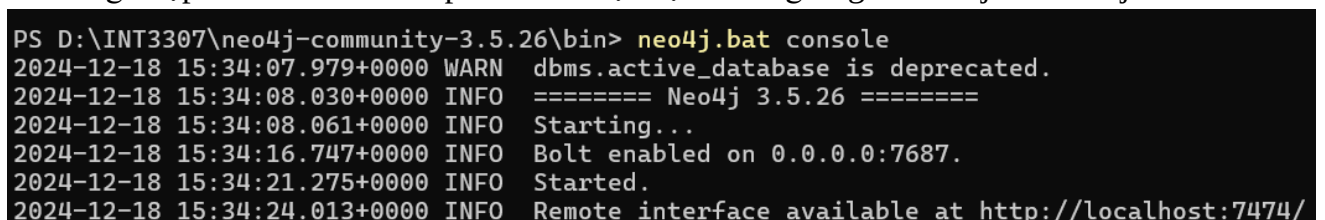
Do đó, cần xác định cơ sở dữ liệu nào cần được khởi động. Điều này yêu cầu sửa đổi tệp **conf** trong thư mục của cơ sở dữ liệu neo4j. Chỉ khi đó, cơ sở dữ liệu tương ứng mới có thể được khởi động. Ở đây config theo database login:



Hình 15: Config database cho Neo4j

Tiếp theo phải khởi động neo4j bằng cách gõ lệnh **neo4j.bat console**.

Mở **http://localhost:7474/** bằng trình duyệt để xem giao diện tương tác của đồ thị tri thức. Và đăng nhập với username và password mặc định tương ứng là 'neo4j' và 'neo4j'.



Hình 16: Start Neo4j

Nhập **match (a) return a** tương đương với **select *** của MySQL, có thể được sử dụng để tự kiểm tra.

2.2. Logic của việc tạo đồ thị tri thức

Đầu tiên, run file `MobileKG/GenerateKG/main.py` để bắt đầu generate đồ thị. Các hàm trong mã có chức năng như sau:

- **`analyze()`**: Trích xuất đặc trưng từ báo cáo thử nghiệm ban đầu
- **`connect()`**: Phân tích nội dung của báo cáo thử nghiệm được trích xuất bởi đặc trưng và trích xuất mối quan hệ của nội dung liên quan
- **`generate()`**: Thực hiện phân giải đồng tham chiếu trên dữ liệu được trích xuất từ mối quan hệ và thêm nó vào đồ thị tri thức

Các steps:

- Thay đổi **`graph_type`** trong `'MobileKG/Config/RunConfig.py'` thành chức năng được chỉ định, chẳng hạn như `'login'` hoặc `'airport'`.
- Đặt báo cáo kiểm thử gốc vào thư mục `'MobileKG/Data/Original'` (hoặc vào một thư mục khác và sửa đổi `'original_data_path'` trong `'MobileKG/Config/RunConfig.py'`).
- Dữ liệu của báo cáo phân tích mặc định được lưu trữ trong thư mục `'MobileKG/Data/Analyze'` (hoặc đặt nó vào một thư mục khác và sửa đổi `'analyze_data_path'` trong `'MobileKG/Config/RunConfig.py'`).
- Chạy phương thức `'analyze()'` trong `'MobileKG/GenerateKG/main.py'`, hệ thống sẽ tự động phân tích thông tin hình ảnh và văn bản của báo cáo kiểm thử và lưu vào thư mục được trả bởi `'analyze_data_path'`.
- Các thông tin hình ảnh và văn bản đã được phân tích cần được kết hợp bên dưới. Dữ liệu trung gian kết hợp mặc định được lưu trữ trong thư mục `'MobileKG/Data/Result'` (hoặc đặt trong các thư mục khác và sửa đổi `'connect_data_path'` trong `'MobileKG/Config/RunConfig.py'`).
- Chạy phương thức `'connect()'` trong `'MobileKG/GenerateKG/main.py'`, hệ thống sẽ tự động liên kết dữ liệu đã được phân tích trong bước trước, tạo một thư mục mới với tên tệp là thời gian hiện tại và dữ liệu trung gian cũng sẽ được lưu trong thời gian hiện tại trong thư mục có tên tệp. Thư mục này sẽ được lưu vào thư mục được trả bởi `'connect_data_path'`.
- Chạy neo4j từ command line.
- Sửa đổi `'generate_data_path'` trong `'MobileKG/Config/RunConfig.py'` và đặt giá trị thành đường dẫn của thư mục được tạo với tên tệp là thời gian hiện tại được đề cập trong bước trước.
- Chạy phương thức `'generate()'` trong `'MobileKG/GenerateKG/main.py'`.
- Sau khi thao tác kết thúc, hãy mở URL `'http://localhost:7474/'`, nhập `'match (a) return a'` để xem có đồ thị tri thức được tạo hay không, nếu được tạo thì có nghĩa là việc tạo đã thành công.

2.3. Các bước truy vấn đồ thị

Khi truy vấn đồ thị, chỉ cần gọi hàm `next_step()` trong `MobileKG/SearchKG/main.py`.
`last\_component\_id` là 0, vì đây là ảnh chụp màn hình đầu tiên của điểm chức năng đăng nhập của ứng dụng "Dewu", tức là bước đầu tiên, nó không có nút tiền nhiệm.

`search\_all` được khuyến nghị sử dụng giá trị mặc định, điều này có thể tăng tốc độ tìm kiếm. Giá trị trả về của tìm kiếm như sau, một mảng được trả về, mảng chứa tất cả các thành phần cần được thao tác trên trang.

3. Demo

3.1. Xây dựng đồ thị EKG

Đầu tiên, run hàm `analyze()`:

```
def analyze():  
    print('Begin Transformation')  
    original_path = original_data_path  
    result_path = analyze_data_path  
    trans = FeatureExtract(None, original_path, result_path)  
    trans.execute()  
    print('Transformation Complete')  
    return
```

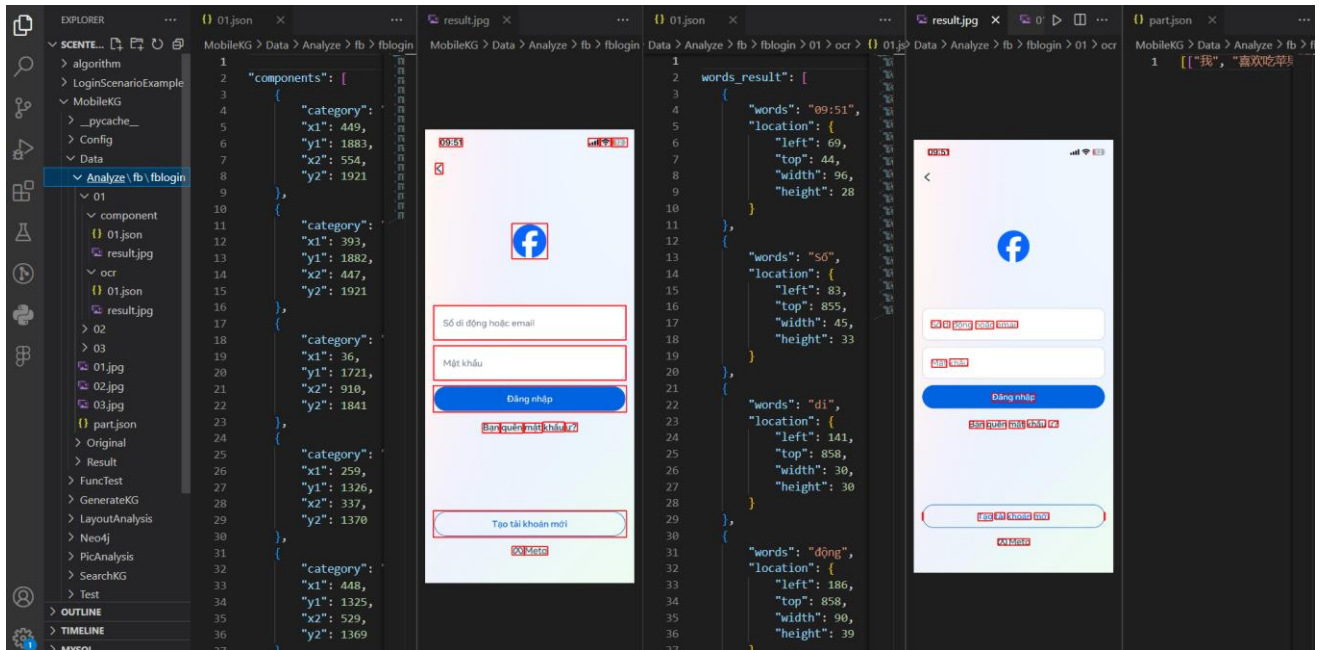
Hình 17: Hàm `analyze()`

```
Model Loaded From D:/INT3307/ScenTest/MobileKG/widAnalysis/vgg16.h5  
Begin Transformation  
D:/INT3307/ScenTest/MobileKG/Data/Original/fb/fblogin/01.jpg  
Complete component extraction in 0.11 seconds  
Complete component extraction in 1.59 seconds  
Complete component extraction in 0.00 seconds  
D:/INT3307/ScenTest/MobileKG/Data/Original/fb/fblogin/02.jpg  
Complete component extraction in 0.10 seconds
```

Hình 18: Quá trình run hàm `analyze()`

Kết quả, ta sẽ được 1 folder chứa các phân tích từ hình ảnh và mô tả:

- Phân tích các thành phần widget
- Phân tích nhận diện chữ OCR
- Phân tích NLP từ file mô tả



Hình 19: Kết quả sau khi run hàm `analyze()`

Tiếp theo, run hàm `connect()`:

```
def connect():
    print('Begin Connection')
    root = analyze_data_path + '/'
    dirs = []
    apps = os.listdir(root)
    for app in apps:
        path = os.listdir(root + app)
        for p in path:
            dirs.append(root + app + '/' + p)
    # dirs=[root+'MaoYan/MaoYan-01']
    c = RelationExtract(dirs, ocr_similarity, opt_similarity,
                        operation_input_similarity, connect_data_path, generate_supply_path)
    c.execute()
    print('Connection Complete')
```

Hình 20: Hàm `connect()`

```
To enable the following instructions: SSE SSE2 SSE3 SSE4.1 SSE4.2 AVX
compiler flags.
Model Loaded From D:/INT3307/ScenTest/MobileKG/WidAnalysis/vgg16.h5
Begin Connection
Connection Complete
```

Hình 21: Quá trình run hàm `connect()`

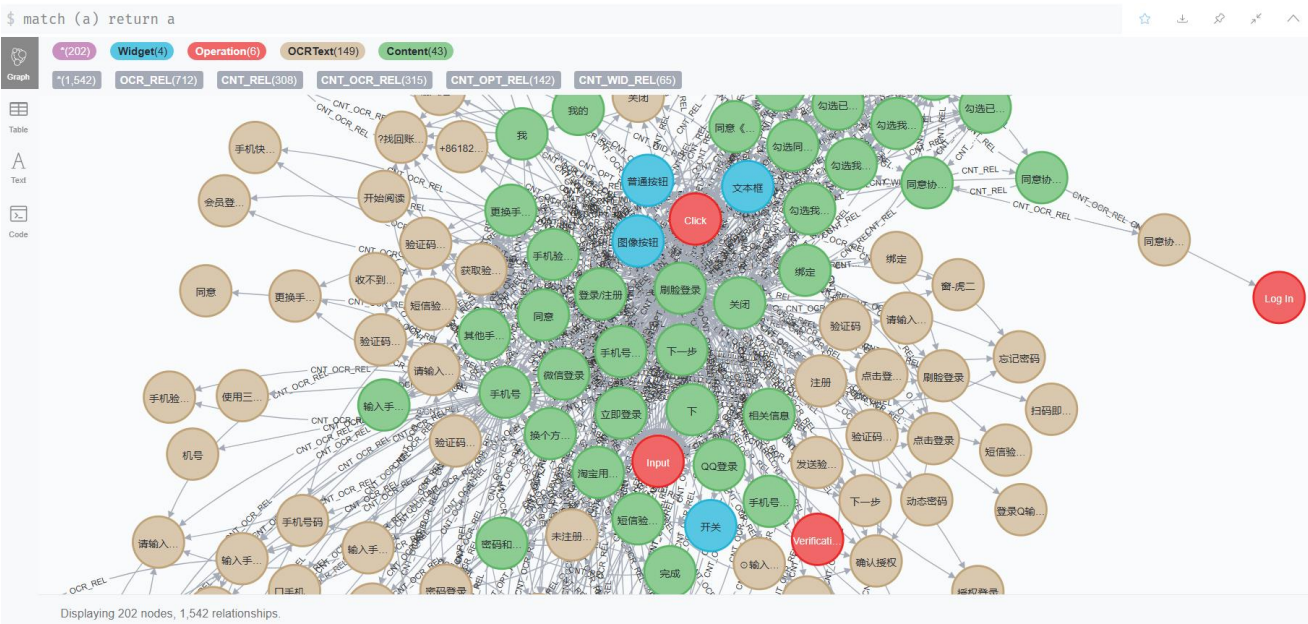
Cuối cùng run hàm `generate()`

```
def generate():
    print('Begin KG Generation')
    gen = GenerateGraph(generate_data_path)
    gen.execute()
    print('KG Generation Complete')
    return
```

Hình 22: Hàm *generate()*

```
Model Loaded From D:/INT3307/ScenTest/MobileKG/widAnalysis/vgg16.h5
Begin KG Generation
Graph Add: match (cnt:Content) where cnt.id=1
match (wdg:Widget) where wdg.id=5
create (cnt)-[r:CNT_WID_REL{relation:"widget"}]->(wdg)
Graph Add: match (a:Content) where a.id=1 set a.head="True"
Graph Add: match (cnt:Content) where cnt.id=2
match (wdg:Widget) where wdg.id=4
create (cnt)-[r:CNT_WID_REL{relation:"widget"}]->(wdg)
```

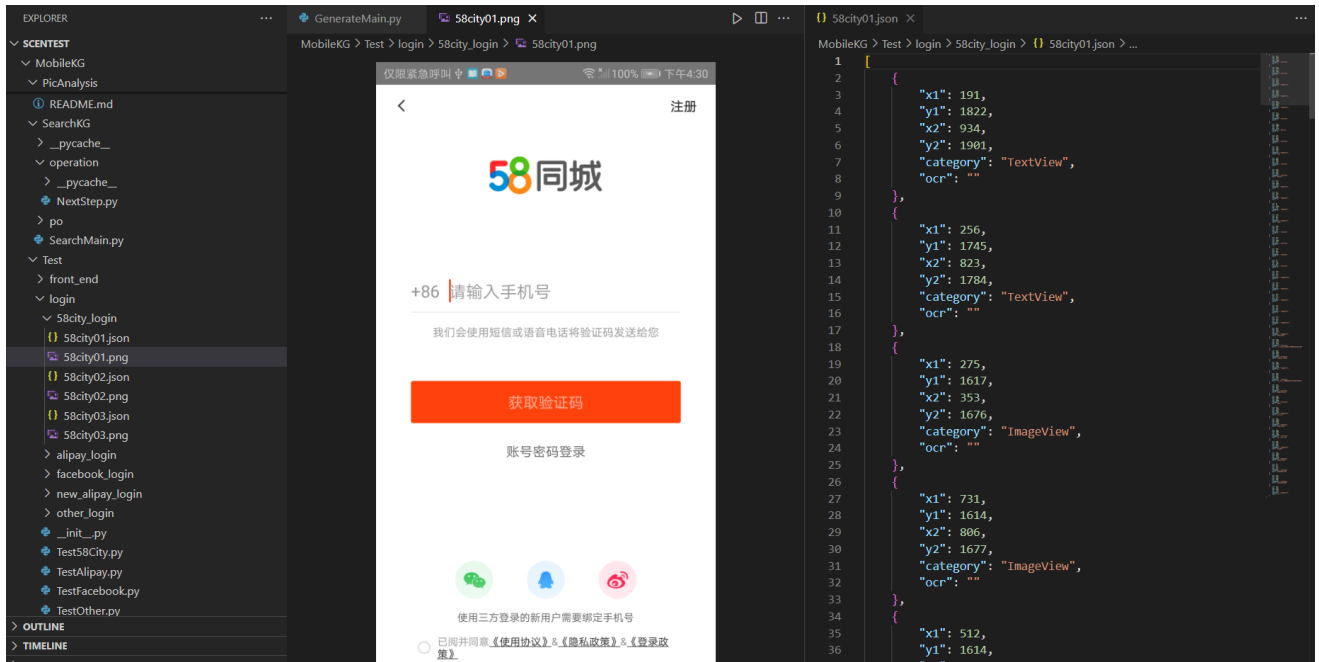
Hình 23: Quá trình run hàm `generate()` – Thêm node vào đồ thị



Hình 24: Kết quả xây dựng cây đồ thị EKG

3.2. Kiểm thử ứng dụng di động qua hình ảnh

Giả sử, cho input là hình ảnh màn đăng nhập và thành phần widget trong đó



Hình 25: Ví dụ về state ban đầu muốn test

Tiếp theo, run để test hình ảnh trên.

```
path = 'D:/INT3307/ScenTest/MobileKG/Test/login/'

def test_pic_1():
    picture = cv2.imread(path + '58city_login/58city01.png', 0)
    split_dic = split(path + '58city_login/58city01.png')
    components = {}
    "components": json.loads(open(path + '58city_login/58city01.json', 'r', encoding='utf-8').read())
}

print('Result for Pic 1:')
print(next_step(picture, split_dic, components, 0, False))
return
```

Hình 26: Hàm test tìm next_steps

Cuối cùng sẽ được các event theo từng steps như này

```
Result for Pic 1:
Click 刷脸登录
Click 同意
Click 关闭
Click 淘宝用户快速登录
Input 手机号
Click 我的
Click 换个方式登录
Click 我
Click 勾选登录注册代表用户同意用户使用协议和隐私政策
Click 手机号登录
```

Hình 27: Kết quả sau khi truy vấn

VII. KẾT LUẬN

SCENTEST là một công cụ kiểm thử ứng dụng di động tự động và hứa hẹn sẽ mang lại nhiều giá trị cho các nhóm phát triển và kiểm thử phần mềm. Bằng việc xây dựng dựa trên kinh nghiệm của người kiểm thử nên công cụ này có khả năng mô phỏng thao tác của người dùng thực. SCENTEST cung cấp một môi trường kiểm thử toàn diện, việc ứng dụng công cụ trong quá trình kiểm thử phần mềm không chỉ cải thiện chất lượng sản phẩm mà còn giúp người kiểm thử tiết kiệm thời gian. Tuy nhiên, bên cạnh những lợi ích đem lại cho người sử dụng thì công cụ này cũng đi kèm với một số hạn chế cần cải thiện.

1. Điểm hay

- Tích hợp kiến thức chuyên môn từ người kiểm thử: SCENTEST trích xuất thông tin từ các báo cáo kiểm thử để làm cơ sở cho việc thực thi nên sẽ bao phủ được nhiều trường hợp liên quan đến logic ứng dụng
- Hỗ trợ kiểm thử nhiều kịch bản phụ nhờ khả năng khám phá các nhánh khác nhau của đồ thị, giúp kiểm thử cả những trường hợp mà kiểm thử thủ công có thể bỏ qua. Điều này giúp phạm vi kiểm thử được bao quát hơn.
- Tự động hóa toàn bộ quá trình kiểm thử từ việc trích xuất dữ liệu cho tới thực thi, giảm thiểu công sức của người kiểm thử phần mềm
- Mô phỏng thao tác trên ứng dụng như người dùng thực, đảm bảo các tình huống thực thi giống với người sử dụng nhất

2. Điểm cần cải thiện

- Cài đặt phức tạp do tích hợp nhiều công cụ ngoài khá nặng, việc cài đặt và sử dụng cũng rất khó khăn. Đặc biệt với những người mới và chưa nhiều kinh nghiệm trong việc cài đặt các môi trường lập trình.
- Tốn thời gian cho những thao tác lặp lại. Vì việc so khớp phải so với từng thực thể trong EKG nên tốn rất nhiều thời gian, đặc biệt là với đồ thị lớn thì việc này có thể làm giảm hiệu suất đáng kể.
- Vì tập tri thức được lấy từ báo cáo nên nó phụ thuộc rất lớn vào lượng báo cáo dùng để xây dựng đồ thị. Nếu lượng báo cáo ít thì tri thức cho hướng dẫn kiểm thử sẽ ít và không đủ để bao phủ được hết những trường hợp kiểm thử. Nhưng nếu lượng báo cáo quá lớn thì việc xây dựng đồ thị là rất tốn thời gian. Không chỉ vậy, một đồ thị quá lớn thì sẽ rất phức tạp trong việc truy vấn và có thể sẽ cần tốn chi phí phần cứng để đáp ứng được nhu cầu vận hành.

Tuy còn những hạn chế nhưng SCENTEST vẫn là một lựa chọn đáng cân nhắc cho các nhóm phát triển cần thử nghiệm những phương pháp mới để tự động hóa việc kiểm thử của mình. Tuy nhiên, để có thể phát huy được hết những lợi ích mà công cụ đem lại thì nhóm phát triển cũng cần chuẩn bị kỹ về kinh nghiệm lập trình để cài đặt và phát triển công cụ cũng như để mở rộng hơn nhằm phù hợp với nhu cầu và dự án.

VIII. PHỤ LỤC

Câu hỏi: Truy vấn là gì? Làm thế nào để có thể truy vấn trong biểu đồ EKG?

Truy vấn (query) là một yêu cầu được sử dụng để tìm kiếm thông tin, dữ liệu từ nguồn dữ liệu database. Với ScenTest, truy vấn được thực hiện trên biểu đồ tri thức sự kiện (EKG – Event Knowledge Graph) để lấy thông tin về các **node** và **step** liên quan. Các node này có thể đại diện cho trạng thái hiện tại hoặc các thành phần của biểu đồ.

Trong mã nguồn, có 2 hàm **search_with_head** và **search_all** đều thực hiện việc truy vấn current state trong biểu đồ EKG để lấy các step tiếp theo:

- **search_with_head():** Truy vấn các bước tiếp theo từ một điểm bắt đầu cụ thể (**head node**).
- **search_all():** Truy vấn toàn bộ các nodes từ biểu đồ, không phụ thuộc vào điểm bắt đầu cụ thể.

Điểm chung của hai hàm đều cùng ý tưởng là:

- So khớp các nodes từ EKG với các components có trong GUI hiện tại.
- Đánh giá mức độ liên quan giữa nodes và components dựa trên các tiêu chí như layout, widget, OCR để chọn ra các node phù hợp nhất.

Để truy vấn dữ liệu từ cơ sở dữ liệu đồ thị (Graph Database) của Neo4j và lấy được danh sách các successor nodes (nút kế tiếp) dựa trên node hiện tại (cnt_id) sẽ dùng hàm **get_next_nodes(cls, cnt_id)**

- Đầu tiên, là tìm các **nodes tương tự** với **node hiện tại** bằng hàm `get_similar_content(cnt_id)` và thêm chính nó vào danh sách.
- Duyệt qua các nodes trong danh sách trên, rồi tìm tất cả các **successor nodes**:

```
MATCH (cnt1:Content)-[r:CNT_REL {relation:"successor"}]->(cnt2:Content)
WHERE cnt1.id={cnt_id}
RETURN cnt2
```
- Tiếp đó, thêm các nodes của các successor nodes vào kết quả cuối cùng.
- Trả về danh sách các successor nodes, bao gồm cả các nodes tương tự.

```

class GraphSearch:

    @classmethod
    def get_next_nodes(cls, cnt_id):
        # 1. Find similar nodes of the current node
        current_similar = GraphSearch.get_similar_content(cnt_id)
        current_similar.append(GraphSearch.get_content(cnt_id))

        # 2. Add all successor nodes of these nodes and remove duplicate nodes
        next_nodes = []
        next_nodes_ids = []
        for c_s in current_similar:
            cypher = 'MATCH (cnt1:Content)-[r:CNT_REL {relation:"successor"}]->(cnt2:Content) WHERE cnt1.id=' + str(
                c_s.id) + ' RETURN cnt2'
            data = GraphSearch.graph.run(cypher).data()
            for i in range(0, len(data)):
                cnt2_id = data[i]['cnt2']['id']
                if cnt2_id not in next_nodes_ids:
                    next_nodes_ids.append(cnt2_id)
                    node = GraphSearch.get_node(cnt2_id)
                    next_nodes.append(node)

        # 3. Add all similar nodes of the successors to the result set
        for n_n in next_nodes:
            similar_contents = n_n.similar
            for s_c in similar_contents:
                if s_c.id not in next_nodes_ids:
                    next_nodes_ids.append(s_c.id)
                    next_nodes.append(GraphSearch.get_node(s_c.id))

        return next_nodes

```

Hình 28: Hàm truy vấn node tiếp theo

TÀI LIỆU THAM KHẢO – NGUỒN

- [1] Practical, Automated Scenario-based Mobile App Testing. [Online]. Available: <https://arxiv.org/pdf/2406.08340>
- [2] Hình 1: Sự phát triển của ứng dụng di động. Source: Mobile app downloads worldwide from 2021 to 2026, by store. Available: <https://www.statista.com/statistics/1010716/apple-app-store-google-play-app-downloads-forecast/>
- [3] Hình 5: Ví dụ đồng tham chiếu. Source: What is Coreference Resolution in NLP?. Available: <https://spotintelligence.com/2024/01/17/coreference-resolution-nlp/>
- [4] 12 thống kê hàng đầu về Cửa hàng Google Play 2024 Bạn Phải Biết. Available: <https://support.google.com/googleplay/thread/269172961?hl=vi>